

Zero Cost Scripting Languages for Game Engines With C++26 Static Reflection

KOEN SAMYN

Koen Samyn

Program coordinator Game Development
Digital Arts and Entertainment

Board member of BeCPP

Author of “Mathematics for Game Creators”



C++ as a compilation host for other languages

- **The vision**

Compile scripting languages and DSLs into native code through C++

C++ as a compilation host for other languages

- **The vision**

Compile scripting languages and DSLs into native code through C++

- **The approach**

Transform AngelScript bytecode into C++ template specializations at compile time, using C++26 reflection

C++ as a compilation host for other languages

- **The vision**

Compile scripting languages and DSLs into native code through C++

- **The approach**

Transform AngelScript bytecode into C++ template specializations at compile time, using C++26 reflection

- **What you'll learn**

Generate specializations with `std::meta::substitute`

Compose instruction calls with fold expressions

See how the compiler can eliminate VM overhead

Part 0 - why

Game Engines and script languages



UNREAL
ENGINE

Game Engines and script languages



**UNREAL
ENGINE**

Blueprints

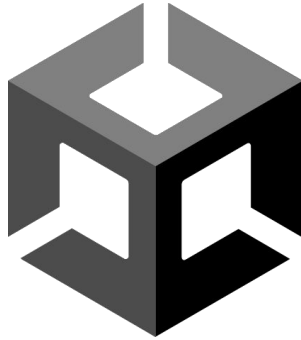
C++

Verse (new)

Game Engines and script languages



**UNREAL
ENGINE**



Unity

Blueprints

C++

Verse (new)

Game Engines and script languages

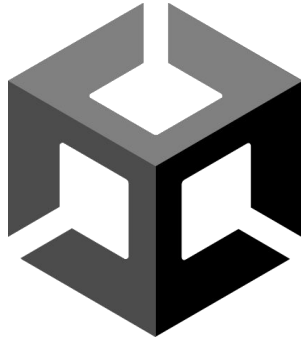


**UNREAL
ENGINE**

Blueprints

C++

Verse (new)



Unity

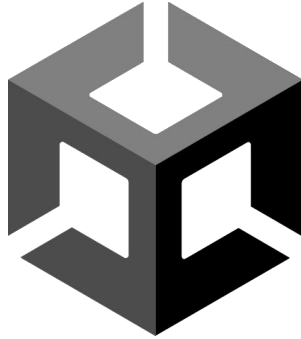
C#

Game Engines and script languages



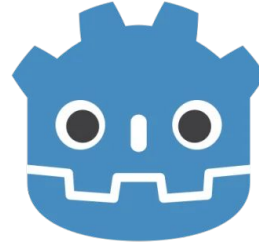
UNREAL
ENGINE

Blueprints
C++
Verse (new)



Unity

C#



GODOT
Game engine

GDScript

Why AngelScript?

Compiles to
bytecode

Why AngelScript?

Compiles to
bytecode

Typed
Language

Why AngelScript?

Compiles to
bytecode

Typed
Language

Has an
emulator

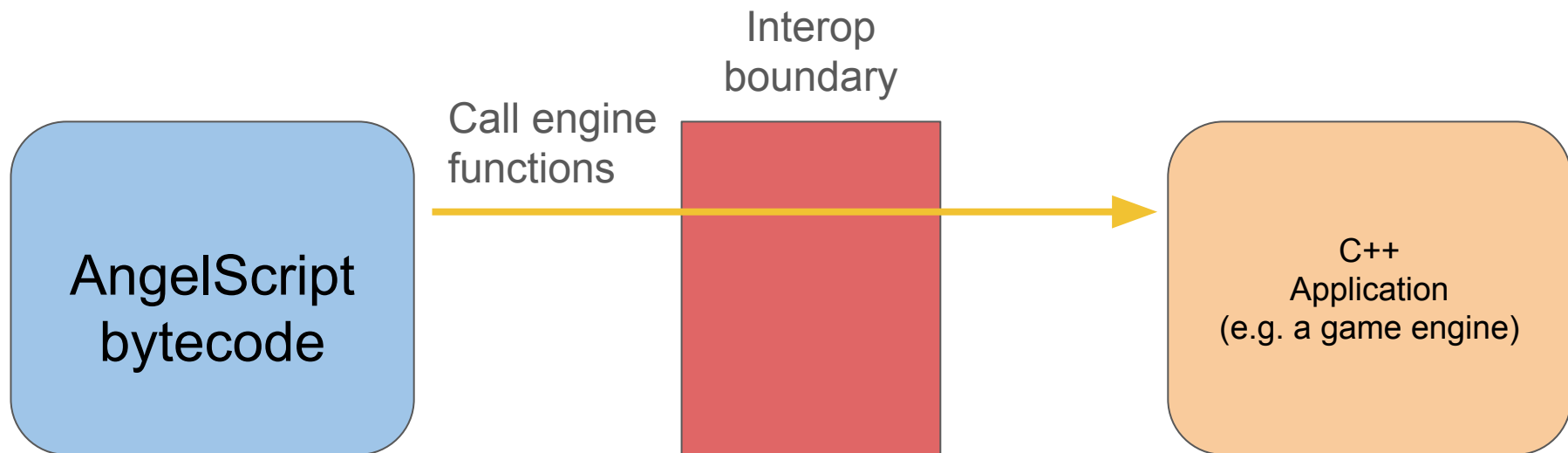
Why AngelScript?

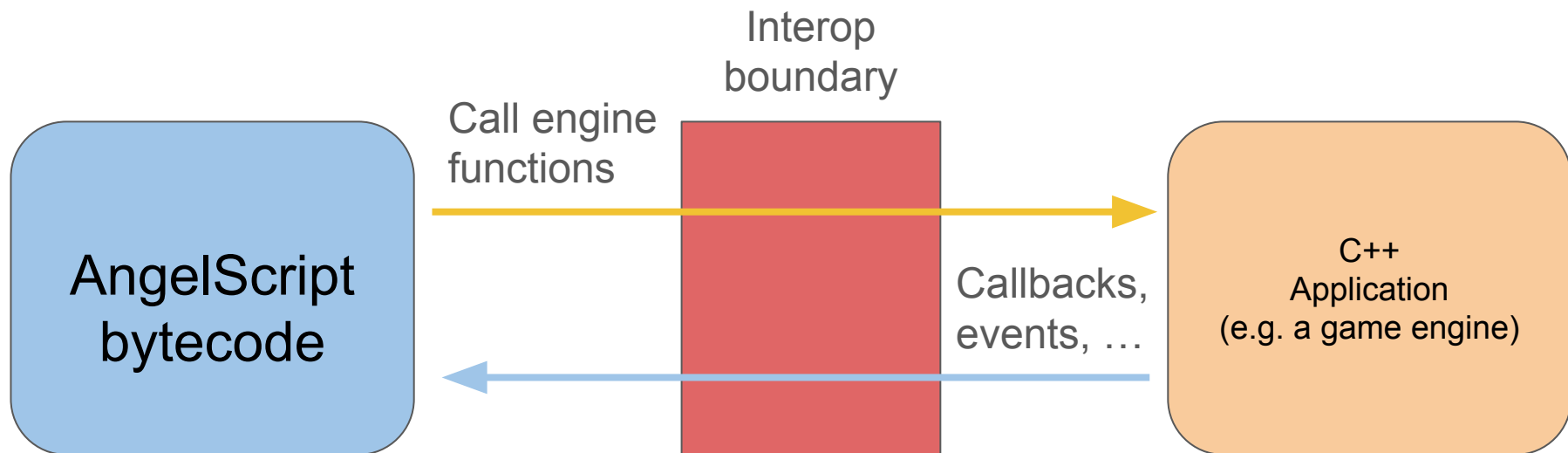
Compiles to
bytecode

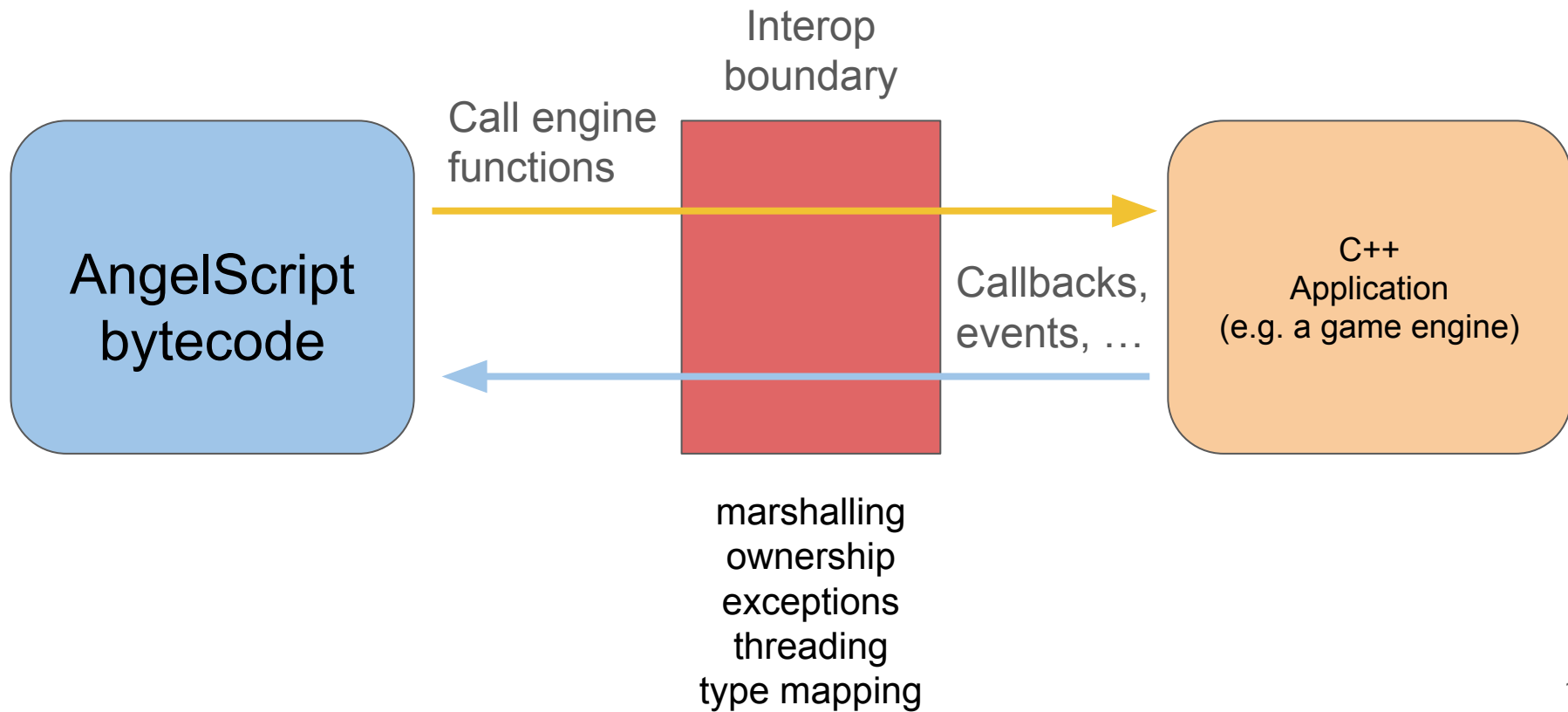
Typed
Language

Has an
emulator

(And it looks a lot like C++)







C++ Application (e.g. a game engine)

```
void utils::drawEllipse(){  
    // ...  
}
```

AngelScript bytecode

```
void draw()  
{  
    drawEllipse();  
}
```

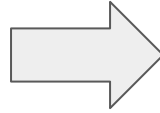
Part 1 - it all adds up

A simple AngelScript program

```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```

A simple AngelScript program

```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```



AngelScript
Bytecode

A simple AngelScript program

```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```

SUSPEND

SetV4 1, 2

A simple AngelScript program

```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```

SUSPEND

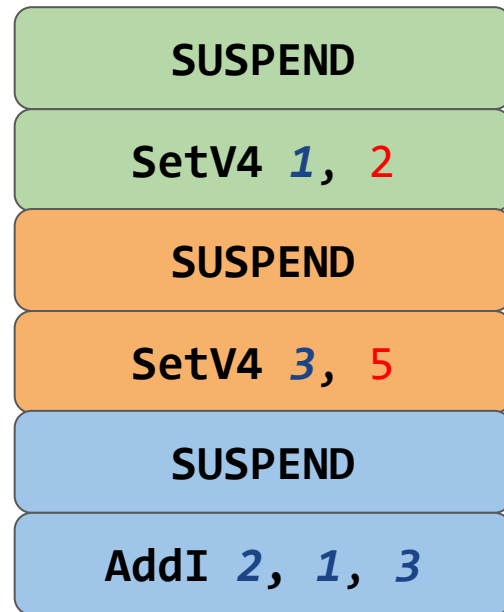
SetV4 1, 2

SUSPEND

SetV4 3, 5

A simple AngelScript program

```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```

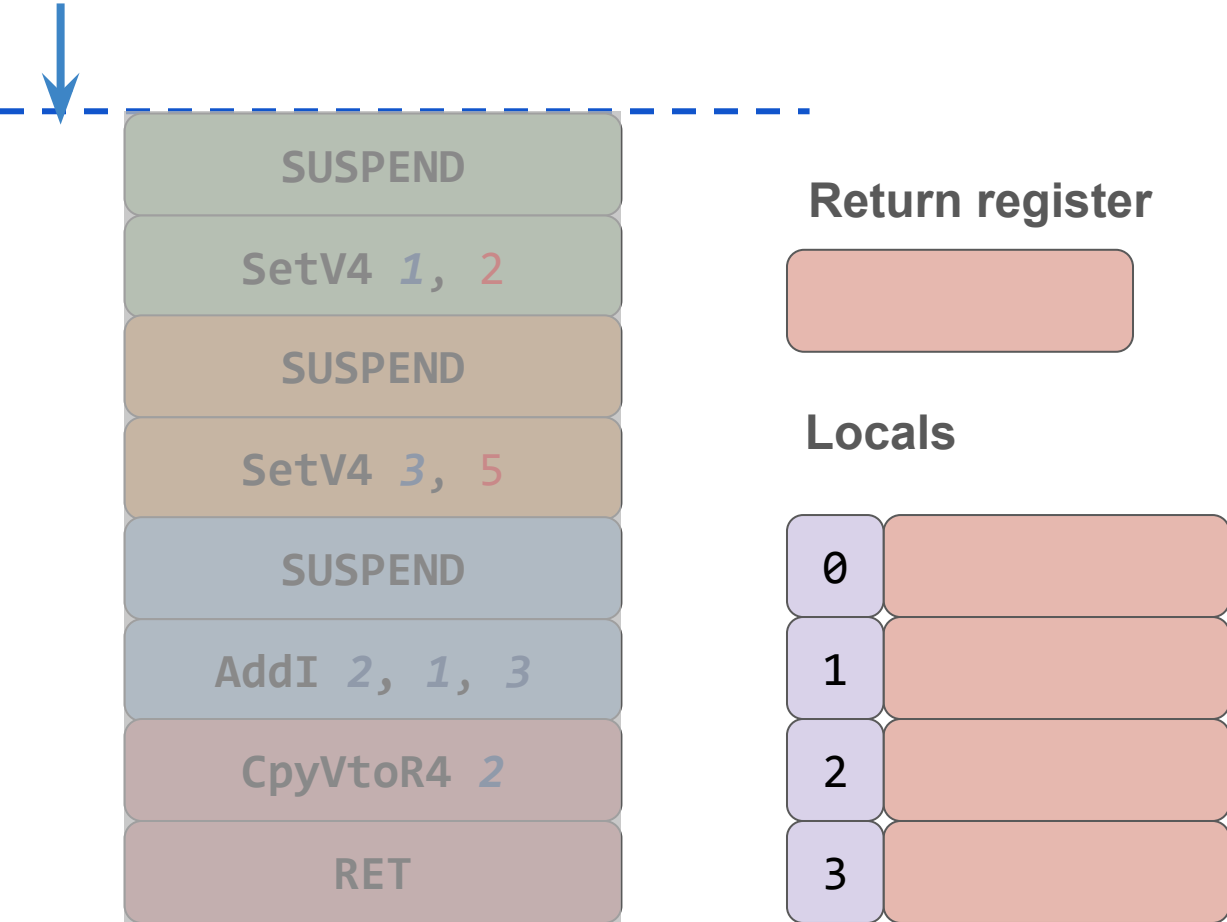


A simple AngelScript program

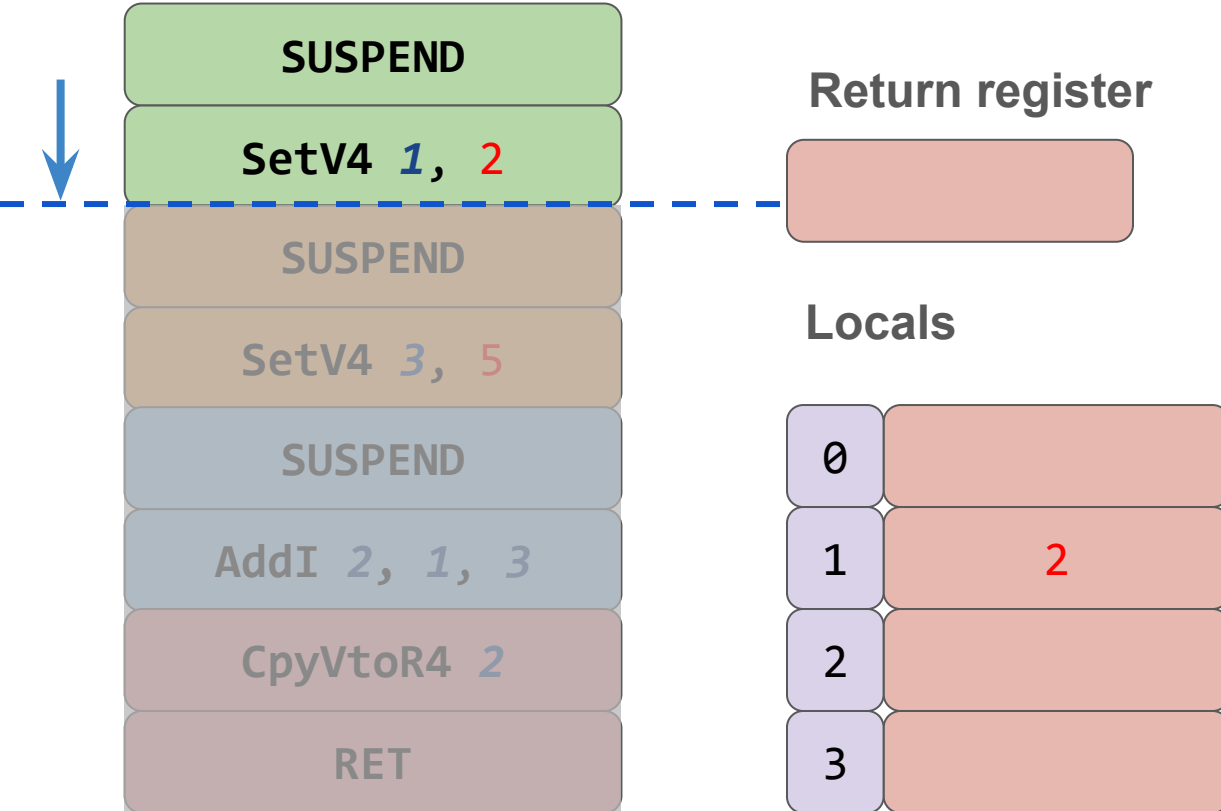
```
int sum()  
{  
    int op1 = 2;  
    int op2 = 5;  
    int sum = op1 + op2;  
    return sum;  
}
```



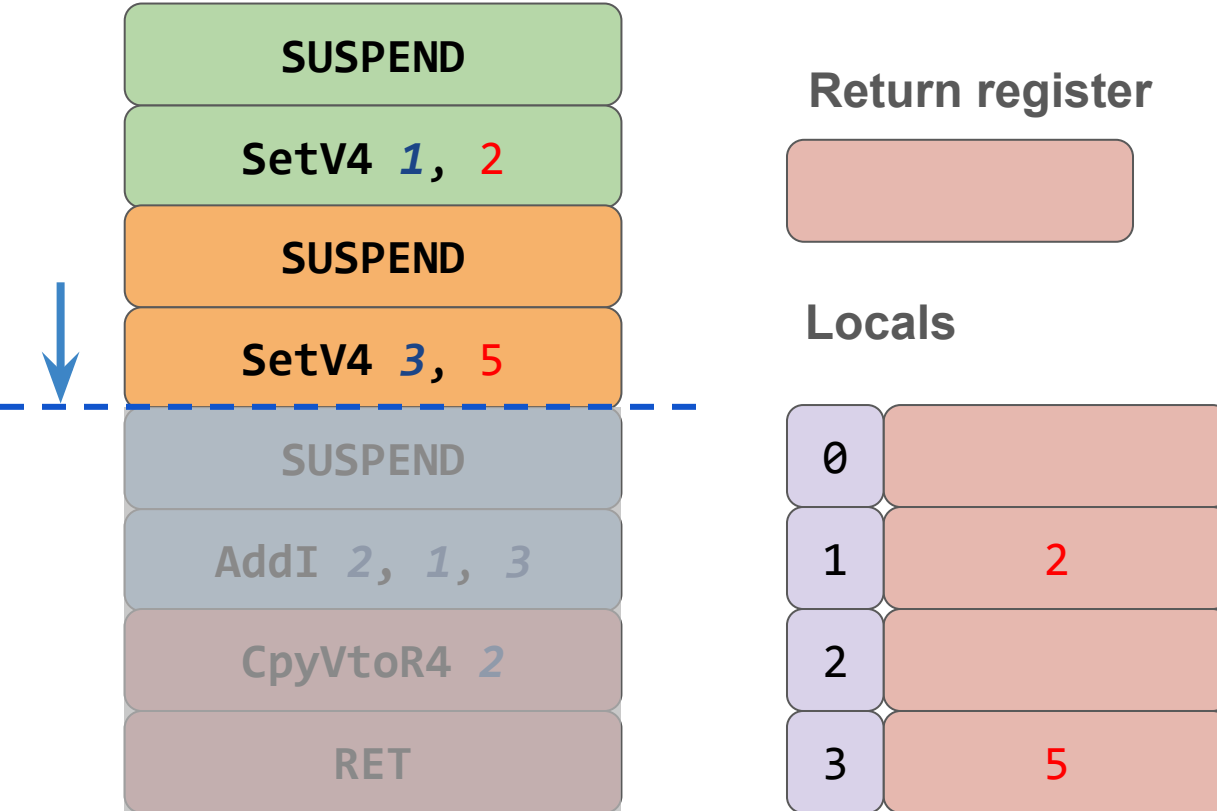
Bytecode execution



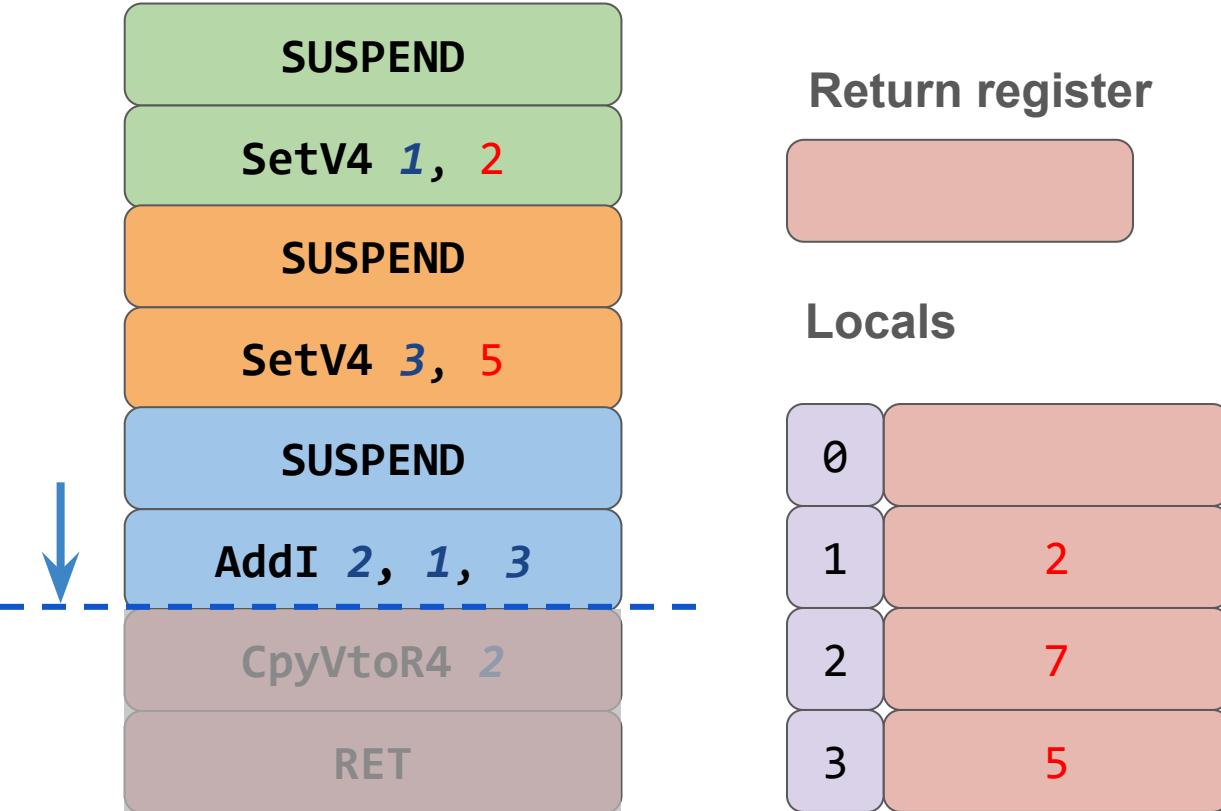
Bytecode execution



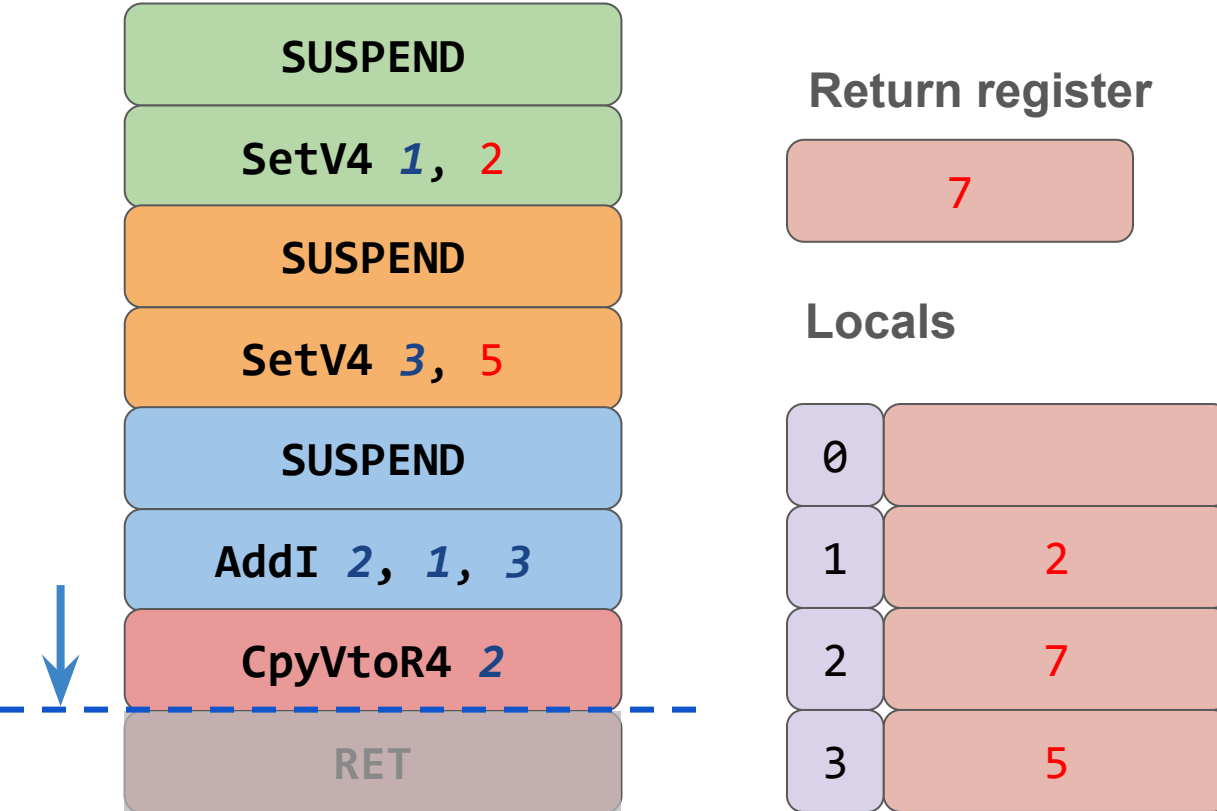
Bytecode execution



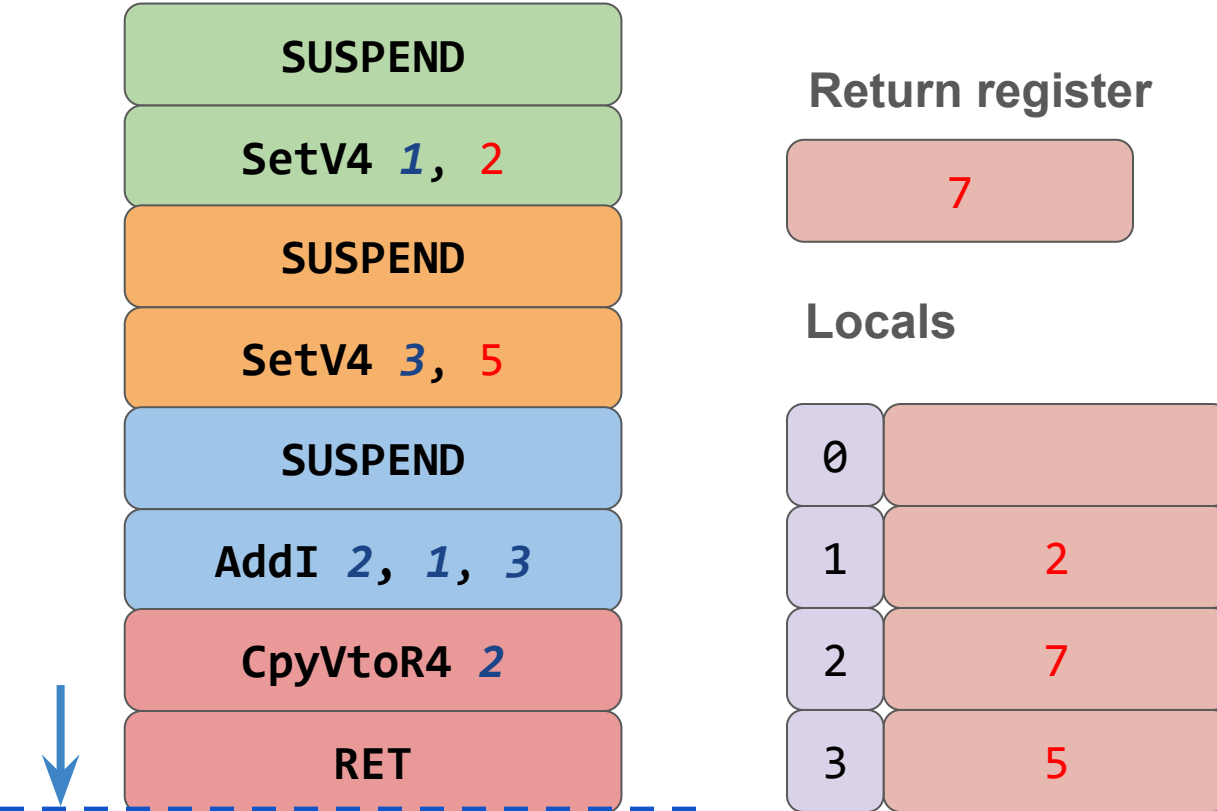
Bytecode execution



Bytecode execution



Bytecode execution



Part 2 - Interpret the bytecode in C++

The emulator approach

Emulation principle

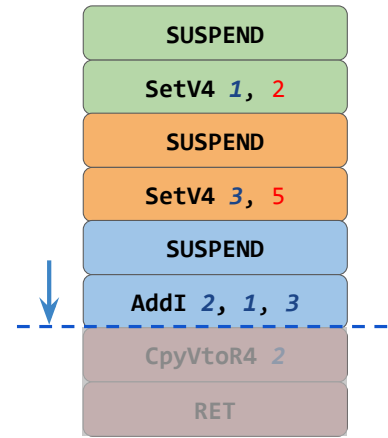
```
using dword std::uint32_t;  
using index std::int16_t;  
struct Frame {  
    std::array<dword, 4> locals{};  
  
};
```

Locals (dword)

0	
1	2
2	7
3	5

Emulation principle

```
using dword std::uint32_t;
using index std::int16_t;
struct Frame {
    std::array<dword, 4> locals{};
    index programCounter{0};
};
```

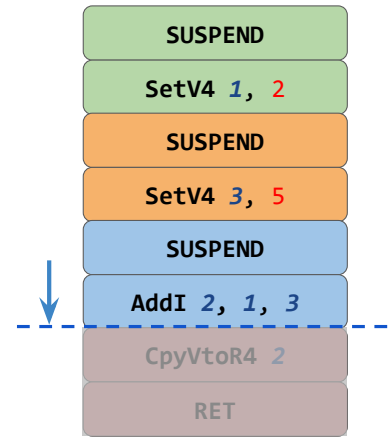


Locals (dword)

0	
1	2
2	7
3	5

Emulation principle

```
using dword std::uint32_t;
using index std::int16_t;
struct Frame {
    std::array<dword, 4> locals{};
    index programCounter{0};
    dword R4{};
};
```



Locals (dword)

0	
1	2
2	7
3	5

Get local variable

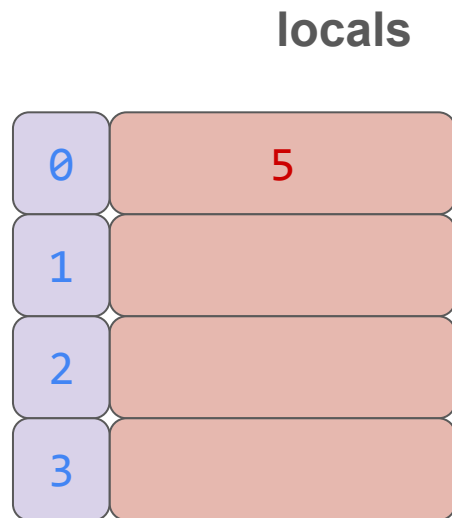
```
template<typename ValueType>
ValueType Frame::get(index i) const
{
    return std::bit_cast<ValueType>( locals[i] );
}
```

locals

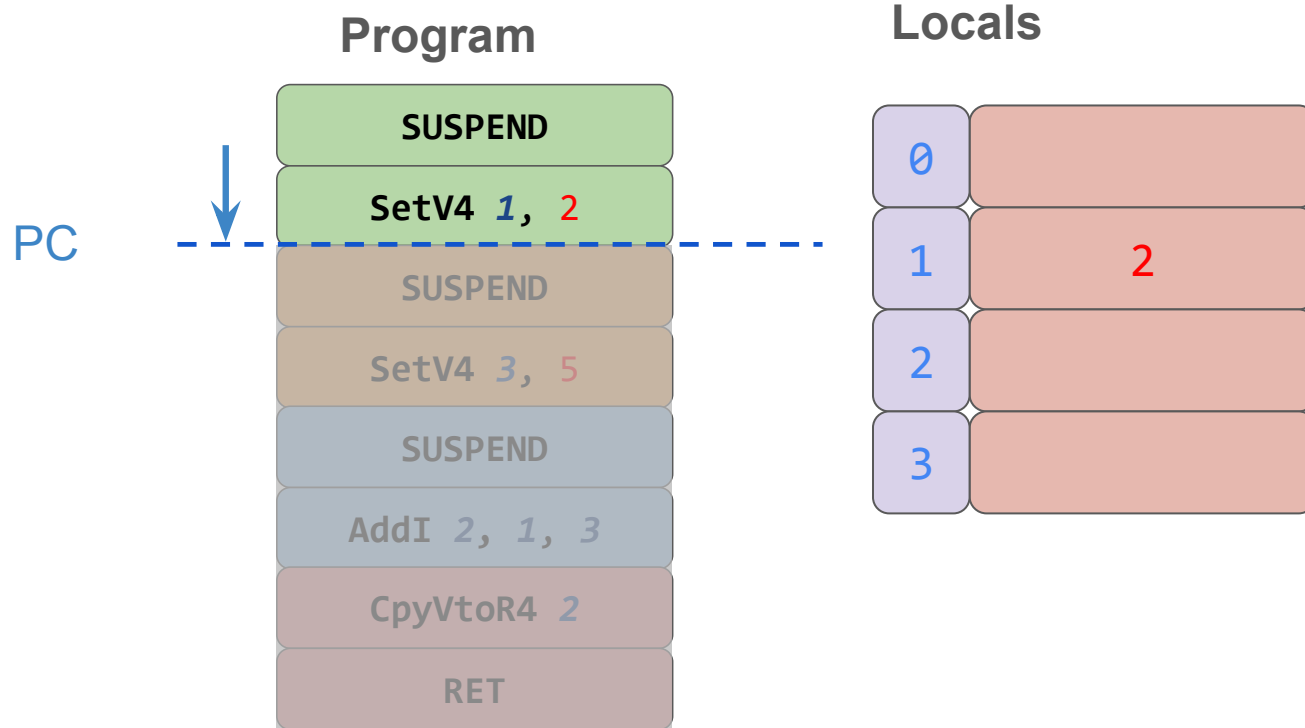
0	5
1	
2	
3	

Set local variable

```
template<typename ValueType>
void Frame::set(index i, ValueType value )
{
    locals[i] = value;
}
```



Frame



Execute function (emulation)

```
void execute(Frame& frame, ByteCode& bc)
{
    switch(op){

        case SUSPEND:
            // do nothing
            break;

        // ...
    }
}
```

Execute function (emulation)

```
void execute(Frame& frame, ByteCode bc)
{
    // ...

    case SetV4:
        dword value = joinOperandWords(bc.arg1, bc.arg2);
        frame.set(bc.arg0, value);
        break;

    // ...
}
```

Execute function (emulation)

```
// ...  
case ADDi:{  
    int32_t op1 = frame.get<int32_t>(bc.arg1);  
    int32_t op2 = frame.get<int32_t>(bc.arg2);  
    frame.set(bc.arg0, op1 + op2);  
    break;  
}  
// ...
```

locals

0	
1	2
2	
3	5

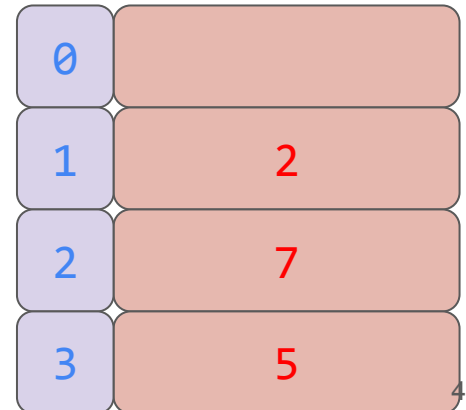
Execute function (emulation)

```
// ...  
    case CpyVtoR4:{  
        frame.R4 = frame.get<dword>(arg0);  
        break;  
    }  
}  
frame.programCounter++;  
}
```

Return register



locals



Main function

```
int main(){
    Frame frame;
    while(frame.programCounter < program.size())
    {
        ByteCode toExecute = program[frame.programCounter];
        execute(frame, toExecute);
    }
    return std::bitcast<int32_t>(frame.R4);
}
```

ASM generation compiler returned: 0

Execution build compiler returned: 0

Program returned: 7

Main function (<https://godbolt.org/z/rEobK9M71>)

```
106 "main":
107     sub     rsp, 56
108     pxor    xmm0, xmm0
109     xor     edx, edx
110     xor     eax, eax
111     mov     DWORD PTR [rsp+32], 0
112     movaps  XMMWORD PTR [rsp], xmm0
113     movaps  XMMWORD PTR [rsp+16], xmm0
114 .L18:
115     movzx   ecx, al
116     mov     rdi, rsp
117     mov     DWORD PTR [rsp+24], edx
118     mov     esi, DWORD PTR "program"[0+rcx*4]
119     mov     BYTE PTR [rsp+32], al
120     call    "execute(VM&, _OpCode)"
121     movzx   eax, BYTE PTR [rsp+32]
122     mov     edx, DWORD PTR [rsp+24]
123     cmp     al, 7
124     jbe     .L18
125     mov     eax, edx
126     add     rsp, 56
127     ret
```

Compiled with GCC 16.1 -O2

Part 2b - Remove the program counter

```
int main(){
    Frame frame;
    for( ByteCode op : program )
    {
        execute(frame, op);
    }
    return frame.valueRegister;
}
```

```
106 "main":
107     sub     rsp, 56
108     pxor    xmm0, xmm0
109     xor     edx, edx
110     xor     eax, eax
111     mov     DWORD PTR [rsp+32], 0
112     movaps  XMMWORD PTR [rsp], xmm0
113     movaps  XMMWORD PTR [rsp+16], xmm0
114     .L18:
115     movzx   ecx, al
116     mov     rdi, rsp
117     mov     DWORD PTR [rsp+24], edx
118     mov     esi, DWORD PTR "program"[0+rcx*4]
119     mov     BYTE PTR [rsp+32], al
120     call    "execute(VM&, OpCode)"
121     movzx   eax, BYTE PTR [rsp+32]
122     mov     edx, DWORD PTR [rsp+24]
123     cmp     al, 7
124     jbe     .L18
125     mov     eax, edx
126     add     rsp, 56
127     ret
```

Part 3 - Templates and if constexpr

Idea :

1. encode the OpCode and arguments as non type template parameters (NTTP) to an execute function

Part 3 - Templates and if constexpr

Idea :

1. encode the OpCode and arguments as non type template parameters (NTPP) to an execute function
2. Instantiate the execute template

Part 3 - Templates and if constexpr

Idea :

1. encode the OpCode and arguments as non type template parameters (NTPP) to an execute function
2. Instantiate the execute template
3. Store the function pointers in
 - a. an array of function pointers

Part 3 - Templates and if constexpr

Idea :

1. encode the OpCode and arguments as non type template parameters (NTPP) to an execute function
2. Instantiate the execute template
3. Store the function pointers in
 - a. an array of function pointers
 - b. a compile-time parameter pack of function pointers

Execute function with `if constexpr`

```
template<OpCode op, index arg0, index arg1, index arg2>
void execute(Frame& vm)
{
    if constexpr(op == SUSPEND) {

    }else if constexpr(op == SetV4) {

    }else if constexpr(op == ADDi) {

    }
    // ...
}
```

Template example

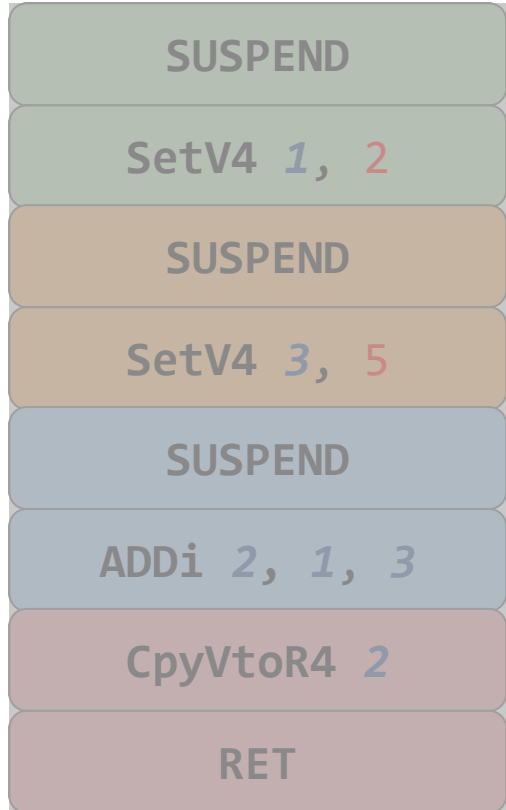
```
template<int32_t op1, int32_t op2>  
int32_t add(){  
    return op1 + op2;  
}
```

Template example

```
template<int32_t op1,int32_t op2>
int32_t add(){
    return op1 + op2;
}
```

```
auto pFn = add<4,3>;
std::cout << "result :" << pFn() << "\n";
```

Handcrafted program



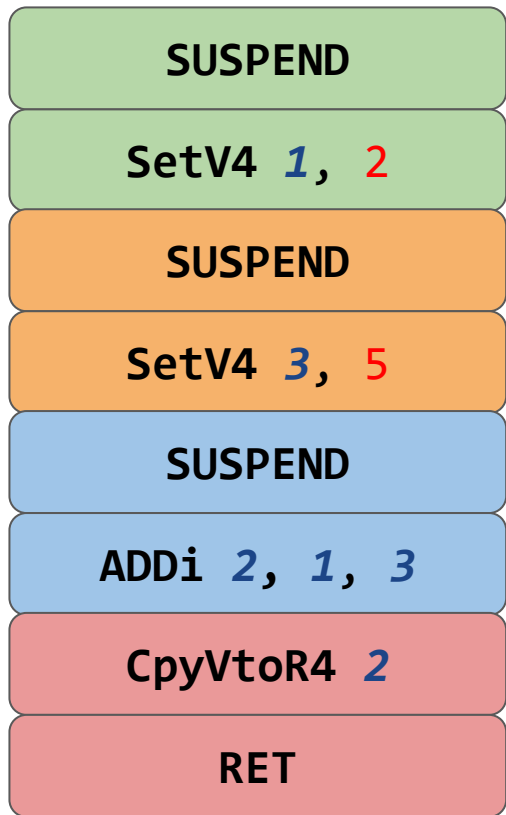
```
int sum(){  
    using InstructionFn = void (*)(VM&);  
    std::array<InstructionFn,8> program = {  
  
    };  
}
```

Handcrafted program



```
int sum(){  
    using InstructionFn = void (*)(VM&);  
    std::array<InstructionFn,8> program = {  
        execute<SUSPEND,0,0,0>,  
        execute<SetV4,1,0,2>,  
        // ...  
    };  
}
```

Handcrafted program



```
int sum(){  
    using InstructionFn = void(*)(VM&);  
    std::array<InstructionFn,8> program = {  
        // ...  
    };  
    Frame f;  
    for(auto instruction:program)  
    {  
        instruction(f);  
    }  
    return std::bit_cast<int32_t>(f.R4);  
}
```

Clang result (-O2)

```
44  sum():  
45      mov     eax, 7  
46      ret  
47  
48  main:  
49      mov     eax, 7  
50      ret
```

Clang result (-O2)

```
44  sum():  
45      mov     eax, 7  
46      ret  
47  
48  main:  
49      mov     eax, 7  
50      ret
```

What is the magic that enables this ?

SRA - scalar replacement of aggregates

```
struct Frame {  
    int stack[8];  
    int sp;  
    int result;  
};
```

```
Frame frame{};  
frame.stack[frame.sp++] = 3;  
frame.stack[frame.sp++] = 4;  
frame.result = frame.stack[--frame.sp]  
               + frame.stack[--frame.sp];  
return frame.result;
```

SRA - scalar replacement of aggregates

```
struct Frame {  
    int stack[8];  
    int sp;  
    int result;  
};
```

SRA

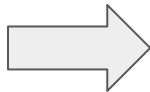


```
Frame frame{};  
frame.stack[frame.sp++] = 3;  
frame.stack[frame.sp++] = 4;  
frame.result = frame.stack[--frame.sp]  
               + frame.stack[--frame.sp];  
return frame.result;
```

```
int a = 3;  
int b = 4;  
int result = a + b;  
return result;
```

GCC Result (-O3)

```
66  "sum()":  
67      push    rbx  
68      sub     rsp, 96  
69      movq    xmm0, QWORD PTR .LC2[rip]  
70      movhps  xmm0, QWORD PTR .LC3[rip]  
71      mov     QWORD PTR [rsp+16], 0  
72      lea     rbx, [rsp+32]  
73      movaps  XMMWORD PTR [rsp+32], xmm0  
74      movq    xmm0, QWORD PTR .LC2[rip]  
75      movhps  xmm0, QWORD PTR .LC4[rip]  
76      movaps  XMMWORD PTR [rsp+48], xmm0  
77      movq    xmm0, QWORD PTR .LC2[rip]  
78      movhps  xmm0, QWORD PTR .LC5[rip]  
79      movaps  XMMWORD PTR [rsp+64], xmm0  
80      movq    xmm0, QWORD PTR .LC6[rip]  
81      movhps  xmm0, QWORD PTR .LC7[rip]  
82      movaps  XMMWORD PTR [rsp+80], xmm0  
83      pxor    xmm0, xmm0  
84      movaps  XMMWORD PTR [rsp], xmm0
```



Not optimized!

Part 3b

Where we get GCC into the fold ...

Compile time sequence

```
using InstructionFn = void (*)(Frame&);
```

```
template<InstructionFn ... Instructions>  
void block(Frame& frame) {  
    (Instructions(frame), ...);  
}
```

Compile time sequence

```
using InstructionFn = void (*)(Frame&);
```

Non Type Template Parameter
pack of function pointers

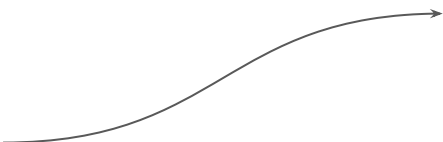


```
template<InstructionFn ... Instructions>  
void block(Frame& frame) {  
    (Instructions(frame), ...);  
}
```

Compile time sequence

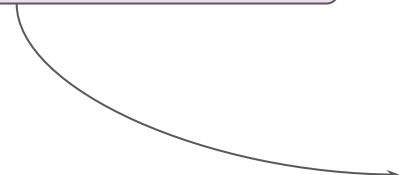
```
using InstructionFn = void (*)(Frame&);
```

Non Type Template Parameter
pack of function pointers

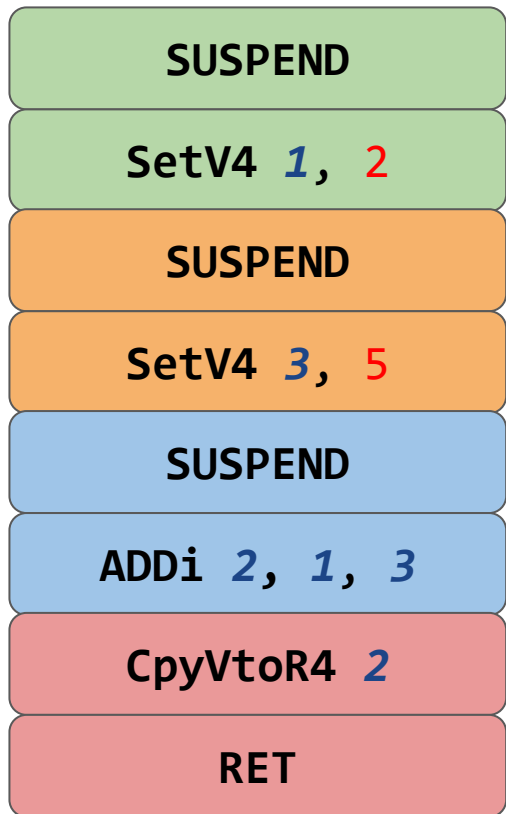


```
template<InstructionFn ... Instructions>  
void block(Frame& frame) {  
    (Instructions(frame), ...);  
}
```

Comma fold expression



Compile time sequence



```
int main(){  
    InstructionFn program = block<  
        execute<SetV4      ,1,0,2>,  
        execute<SetV4      ,3,0,5>,  
        execute<ADDi       ,2,1,3>,  
        execute<CpyVtoR4   ,2,0,0>,  
        execute<RET        ,0,0,0>  
    >;  
  
    Frame frame;  
    program(frame);  
    return frame.R4;  
}
```

Summary so far

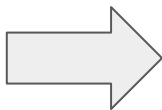
Version	Opcode	Sequence	Clang	GCC
Interpreter	runtime	runtime	✗	✗
Function pointers and for loop	compile-time	runtime	✓	✗
Fold expression	compile-time	compile-time	✓	✓

Part 4 - Reflection

Goal



Reflection



```
int main(){  
    InstructionFn program = block<  
        execute<SetV4      ,1,0,2>,  
        execute<SetV4      ,3,0,5>,  
        execute<ADDi       ,2,1,3>,  
        execute<CpyVtoR4   ,2,0,0>,  
        execute<RET        ,0,0,0>  
    >;  
  
    Frame frame;  
    program(frame);  
    return frame.R4;  
}
```

Reflection operators

^^

Reflection operator

Returns **metadata** from variables or types.

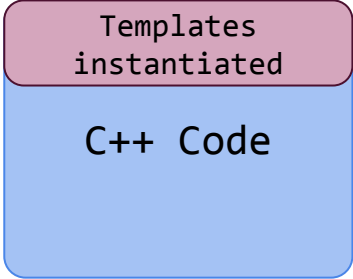
Colloquially: cat-ears operator

[:: ... ::]

Splice operator

Use this **metadata** in a context.

Reflection overview

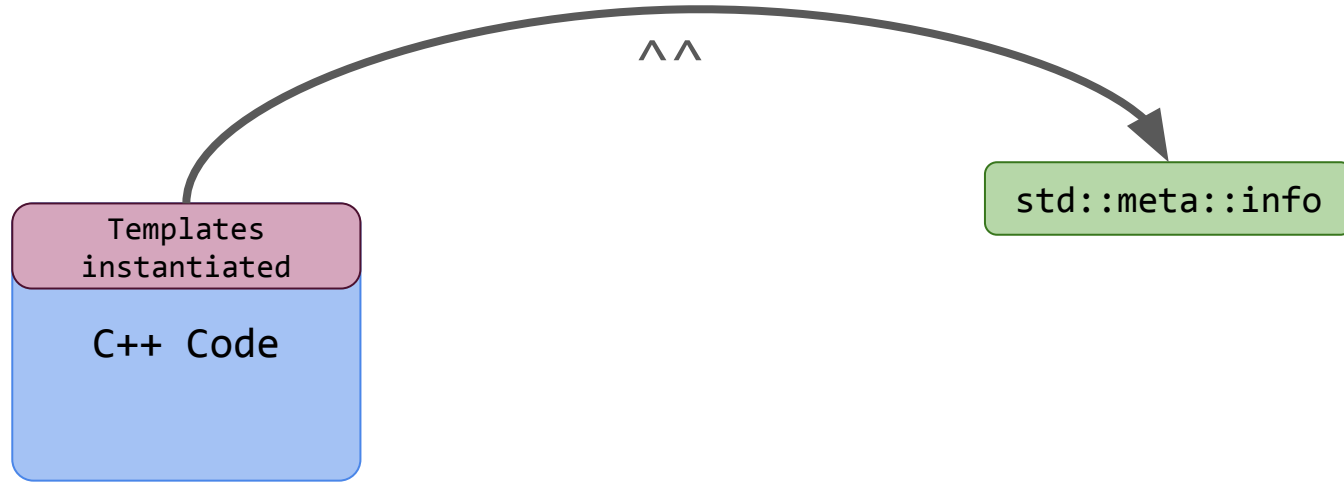


A diagram consisting of two stacked rounded rectangles. The top rectangle is light red and contains the text 'Templates instantiated'. The bottom rectangle is light blue and contains the text 'C++ Code'. The red rectangle is positioned directly above the blue rectangle, indicating that templates are instantiated from the C++ code.

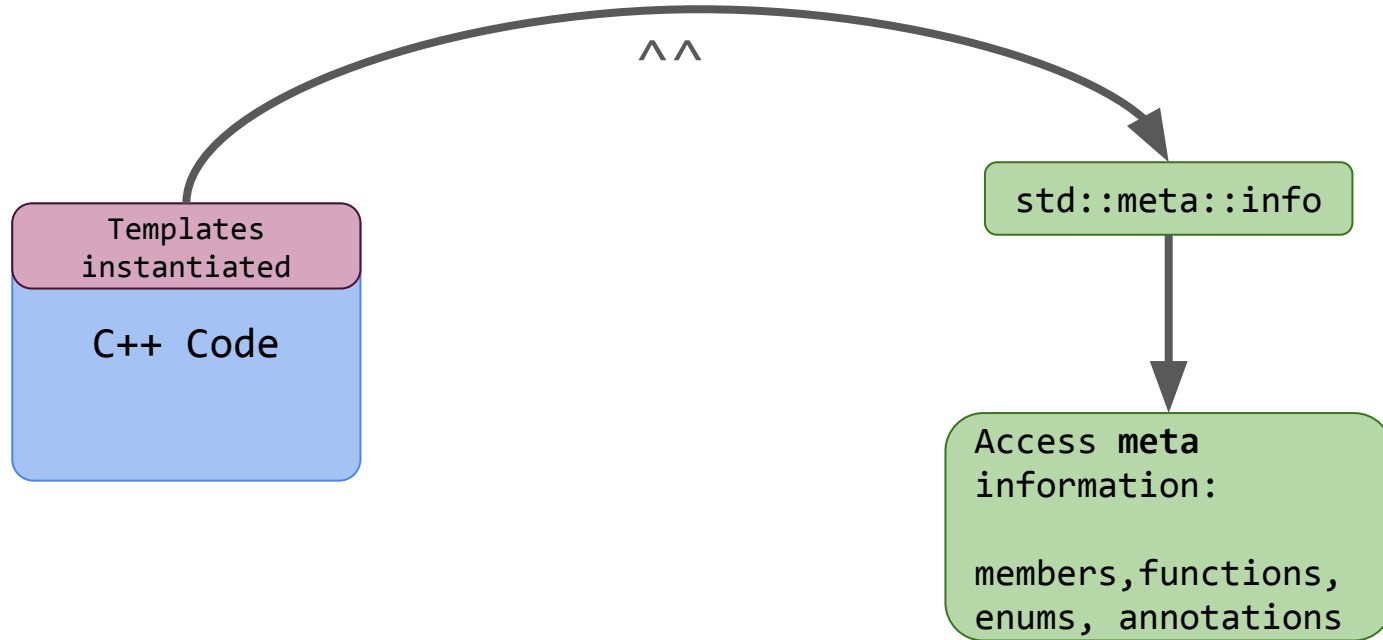
Templates
instantiated

C++ Code

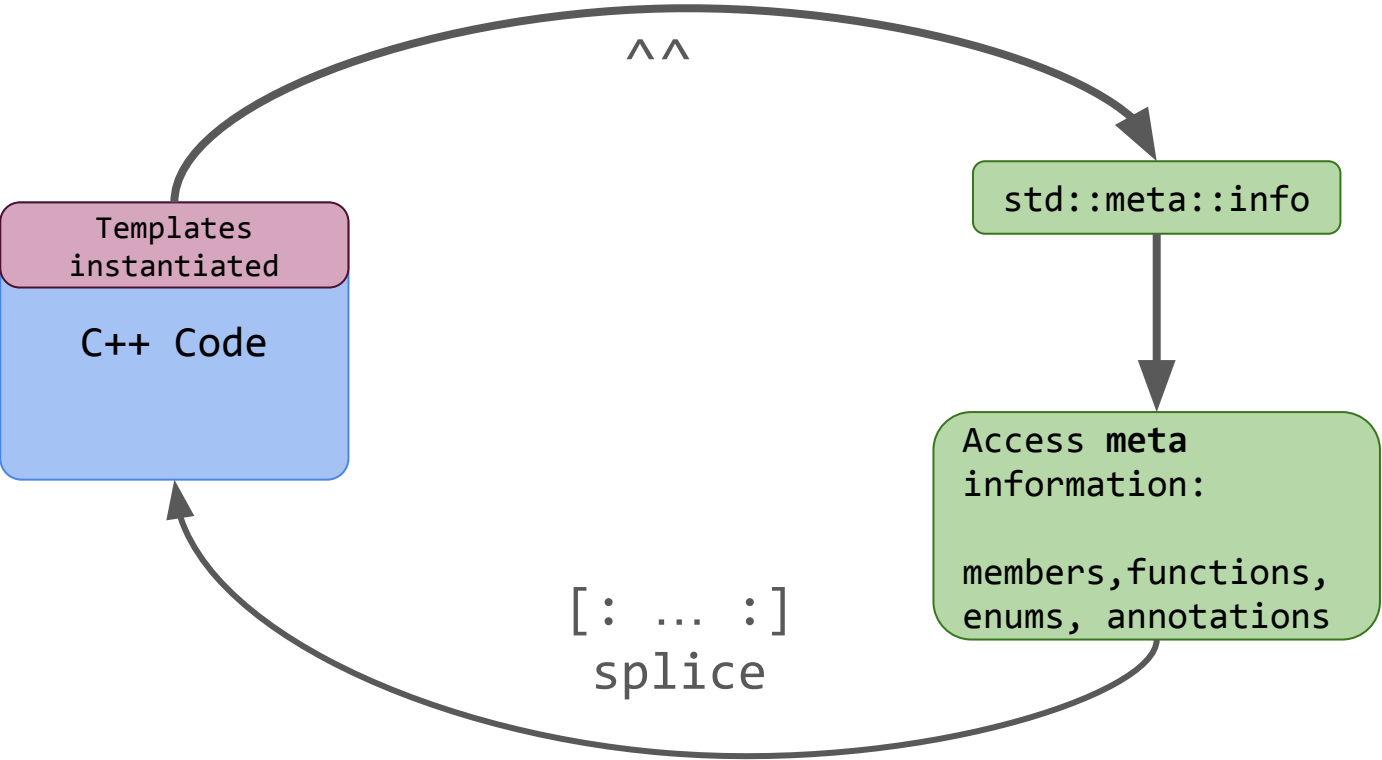
Reflection overview - reflection operator



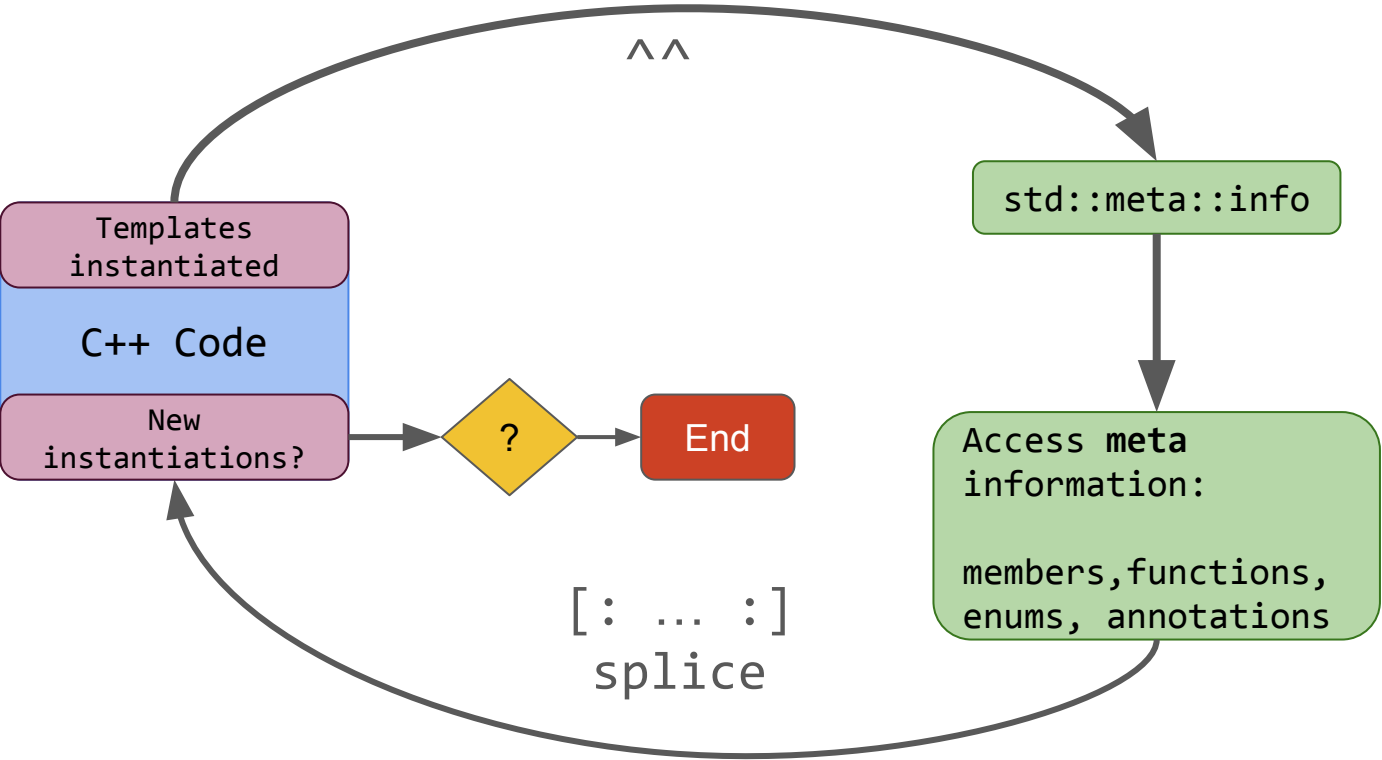
Reflection overview - query functions



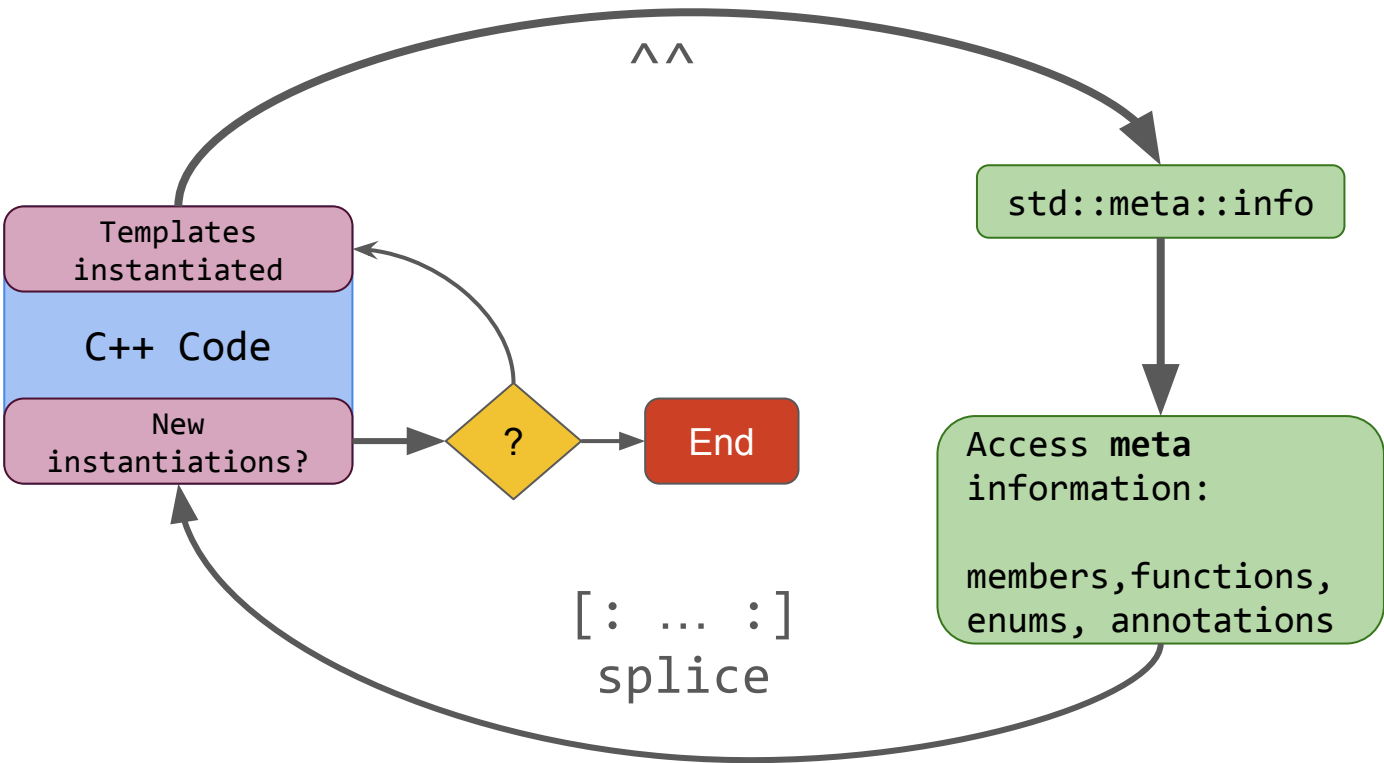
Reflection overview - splice operator



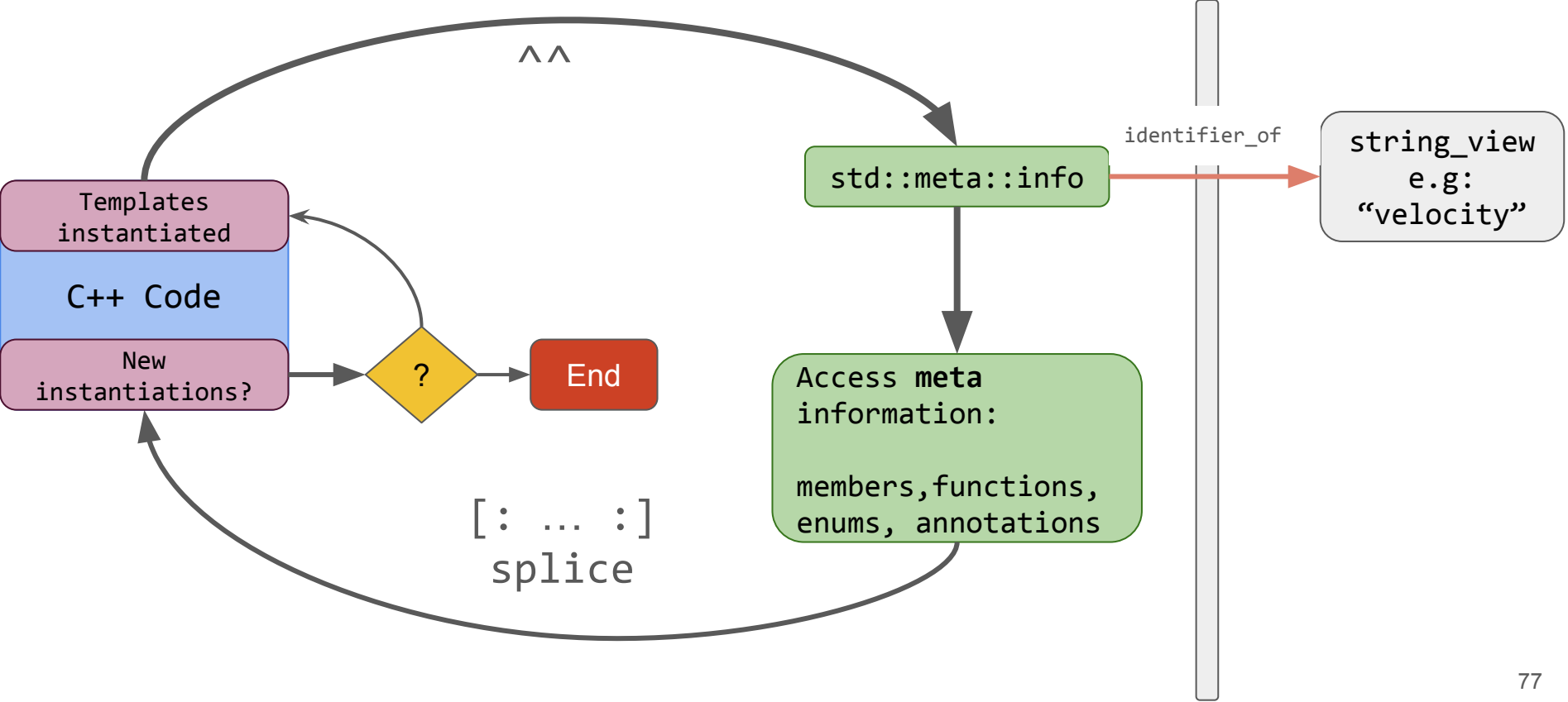
Reflection overview - end of reflection



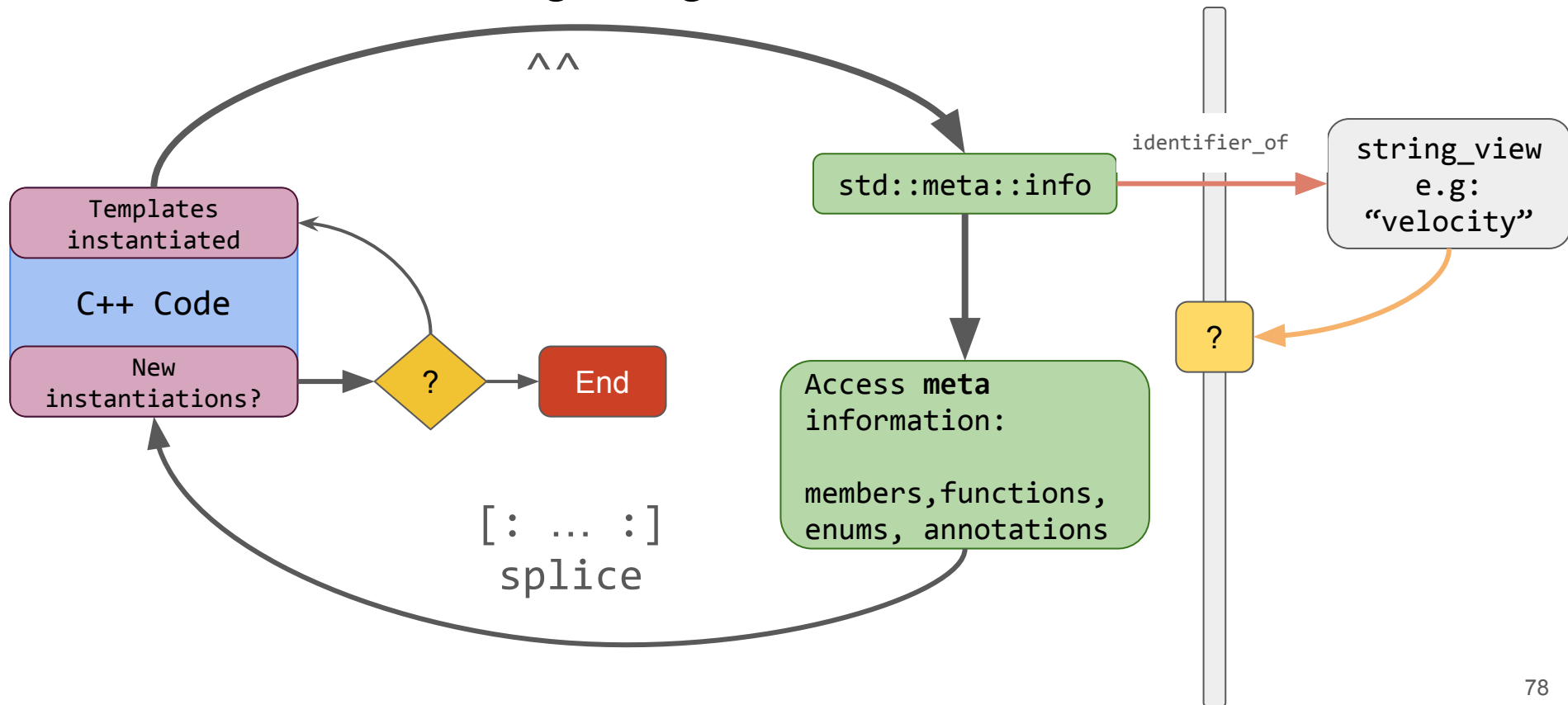
Reflection overview - loop possibility



Reflection overview - escaping out of reflection land



Reflection overview - getting back into reflection land



Let's reflect something

```
template<int A, int B>  
int multiply(){  
    return A*B;  
}
```

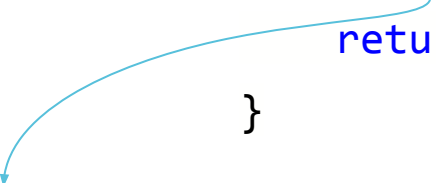
```
int main(){  
    int result = multiply<3, 4>();  
    return result;  
}
```

Let's reflect something

How to reflect the multiply function:

```
std::meta::info executeInfo = ^^multiply;
```

```
template<int A, int B>  
int multiply(){  
    return A*B;  
}
```



Let's reflect something

```
template<int A, int B>  
int multiply(){  
    return A*B;  
}
```

How to reflect the multiply function:

```
constexpr std::meta::info rMultiply = ^^multiply;
```

Reflect the constants so they can be used as template arguments:

```
constexpr std::meta::info rA = std::meta::reflect_constant(3);  
constexpr std::meta::info rB = std::meta::reflect_constant(4);
```

Reflect and finally splice

```
template<int A, int B>  
int multiply(){  
    return A*B;  
}
```

How to reflect the `multiply` function:

```
constexpr std::meta::info rMultiply = ^^multiply;
```

Reflect the constants so they can be used as template arguments:

```
constexpr std::meta::info rA = std::meta::reflect_constant(3);  
constexpr std::meta::info rB = std::meta::reflect_constant(4);
```

Create the reflected specialization:

```
constexpr std::meta::info riMul_3_4 = std::meta::substitute( rMultiply, {rA, rB} );
```

Reflect and finally splice

```
template<int A, int B>  
int multiply(){  
    return A*B;  
}
```

How to reflect the `multiply` function:

```
constexpr std::meta::info rMultiply = ^^multiply;
```

Reflect the constants so they can be used as template arguments:

```
constexpr std::meta::info rA = std::meta::reflect_constant(3);  
constexpr std::meta::info rB = std::meta::reflect_constant(4);
```

Create the reflected specialization:

```
constexpr std::meta::info riMul_3_4 = std::meta::substitute( rMultiply, {rA, rB} );
```

Splice:

```
int result = [: riMul_3_4 :]();
```

How to interpret the splice

```
int result1 = [: riMul_3_4 :] ();
```

as if

```
int result2 = multiply<3,4> ();
```

After the splice, normal C++ rules apply

```
using FnPtr = int(*)();
```

```
FnPtr fp1 = [:riMultiply:];
```

as if

```
FnPtr fp2 = multiply<3,4>;
```

Part 5 - transforming bytecode

Compile time programming time

constexpr

requests that the value of:

- a variable
- structured binding(since C++26)
- or function

is

evaluated at compile-time

enables use in constant expressions

SetV4 1, 2

SetV4 3, 5

ADDi 2, 1, 3

CpyVtoR4 2

RET

InstructionFn pFN = **execute**<ADDi,2,1,3>;

SetV4 1, 2

SetV4 3, 5

ADDi 2, 1, 3

CpyVtoR4 2

RET

~~InstructionFn pFN = execute<ADDi, 2, 1, 3>;~~

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >  
void execute ( Frame& f )  
{  
}
```

^^

std::meta::info

ADDi 2, 1, 3

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >  
void execute ( Frame& f )  
{  
}
```

^^

std::meta::info

ADDi

std::meta::info

ADDi 2, 1, 3

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```

^^

std::meta::info

ADDi

^^

std::meta::info

std::meta::reflect_constant

ADDi 2, 1, 3

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```

^^

std::meta::info

ADDi

^^

std::meta::info

std::meta::reflect_constant

2

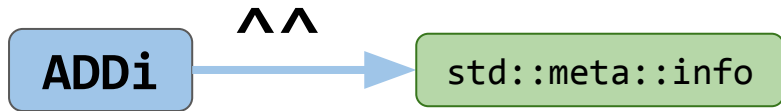
std::meta::info

ADDi 2, 1, 3

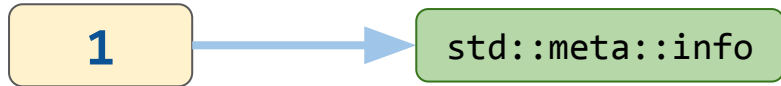
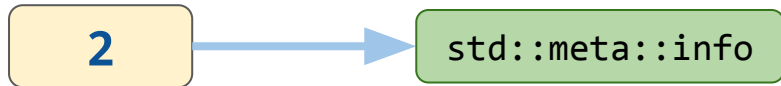
```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```

^^

std::meta::info



std::meta::reflect_constant



ADDi 2, 1, 3

94

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```

^^

std::meta::info

ADDi

^^

std::meta::info

std::meta::reflect_constant

2

std::meta::info

1

std::meta::info

3

std::meta::info

ADDi 2, 1, 3

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```

^^

std::meta::info

ADDi

2

1

3

std::meta::info

std::meta::info

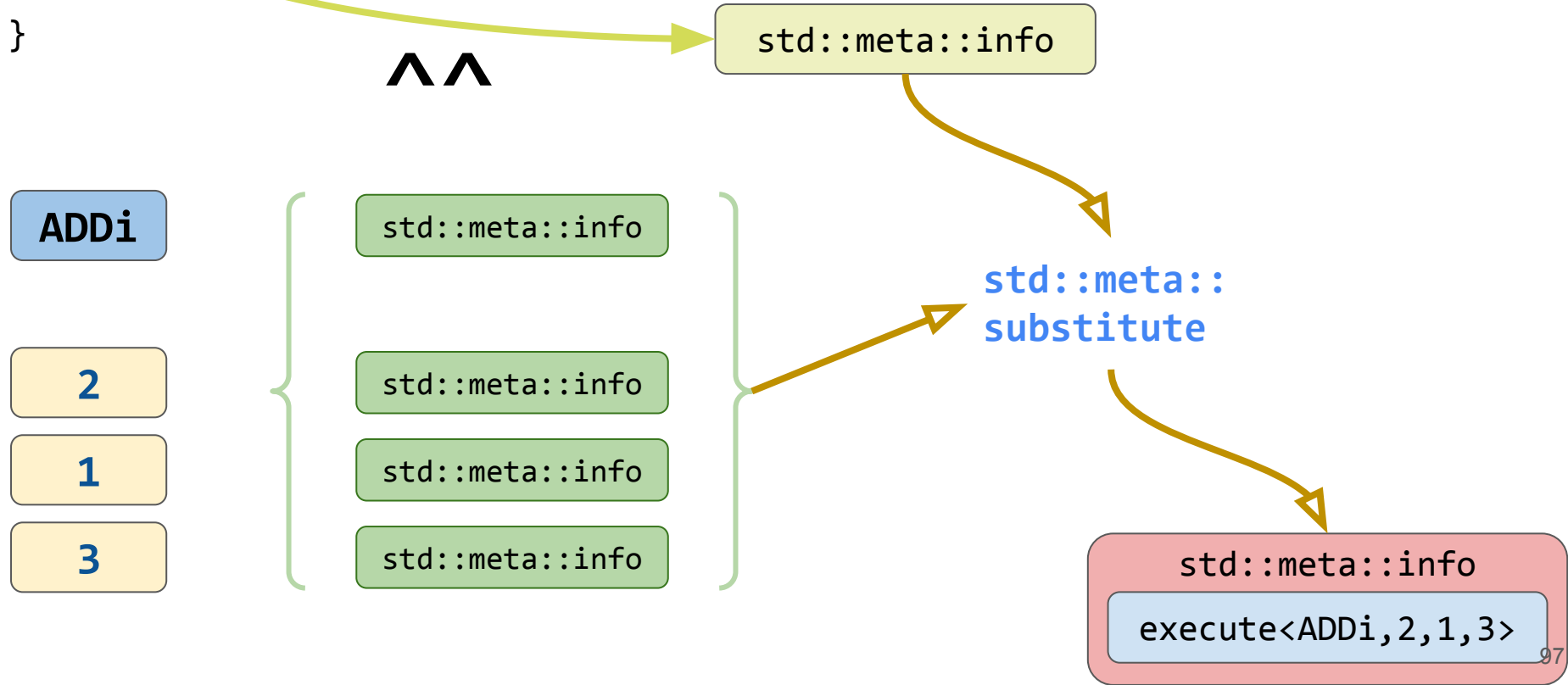
std::meta::info

std::meta::info

std::meta::
substitute

ADDi 2, 1, 3

```
template<OpCode op, int16_t arg0, int16_t arg1, int16_t arg2 >
void execute ( Frame& f )
{
}
```



```
constexpr auto iExecute = ^^execute;
```

```
constexpr auto iExecute = ^^execute;
```

```
constexpr auto opcode = ^^asBC_ADDi;
```

```
constexpr auto dst = std::meta::reflect_constant(2);
```

```
constexpr auto op1 = std::meta::reflect_constant(1);
```

```
constexpr auto op2 = std::meta::reflect_constant(3);
```

```
constexpr auto iExecute = ^^execute;

constexpr auto opcode = ^^asBC_ADDi;
constexpr auto dst = std::meta::reflect_constant(2);
constexpr auto op1 = std::meta::reflect_constant(1);
constexpr auto op2 = std::meta::reflect_constant(3);

constexpr auto pFunction = std::meta::substitute(
    iExecute,
    {opcode, dst, op1, op2}
);
```

```
constexpr auto iExecute = ^^execute;
```

```
constexpr auto opcode = ^^asBC_ADDi;
```

```
constexpr auto dst = std::meta::reflect_constant(2);
```

```
constexpr auto op1 = std::meta::reflect_constant(1);
```

```
constexpr auto op2 = std::meta::reflect_constant(3);
```

```
constexpr auto pFunction = std::meta::substitute(  
    iExecute,  
    {opcode, dst, op1, op2}  
);
```

```
Frame f;
```

```
pFunction(f);
```

```
constexpr auto iExecute = ^^execute;
```

```
constexpr auto opcode = ^^asBC_ADDi;
```

```
constexpr auto dst = std::meta::reflect_constant(2);
```

```
constexpr auto op1 = std::meta::reflect_constant(1);
```

```
constexpr auto op2 = std::meta::reflect_constant(3);
```

```
constexpr auto pFunction = std::meta::substitute(  
    iExecute,  
    {opcode, dst, op1, op2}  
);
```

```
Frame f;
```

```
[ : pFunction :](f);
```

**pFunction is a
std::meta::info object !!**

How to interpret the splice

```
int result1 = [: pFunction :] ();
```

as if

```
int result2 = execute<ADDi,2,1,3> ();
```

After the splice, normal C++ rules apply

```
using FnPtr = int (*)(());
```

```
FnPtr fp1 = [:pFunction:];
```

as if

```
FnPtr fp2 = execute<ADDi,2,1,3>;
```

Substitute a Bytecode program at compile time

```
inline constexpr std::array<dword, 8> sum
```

```
{
```

```
    0x0001004Du,
```

```
    0x00000002u,
```

```
    0x0003004Du,
```

```
    0x00000005u,
```

```
    0x00020073u,
```

```
    0x00030001u,
```

```
    0x00020052u,
```

```
    0x0000000Au
```

```
};
```

Substitute a Bytecode program at compile time

```
inline constexpr std::array<dword, 8> sum
```

```
{
```

```
    0x0001004Du,  
    0x00000002u,
```

```
    0x0003004Du,  
    0x00000005u,
```

```
    0x00020073u,  
    0x00030001u,
```

```
    0x00020052u,  
    0x0000000Au
```

```
};
```

std::meta::
substitute



execute<SetV4,1,0,2>

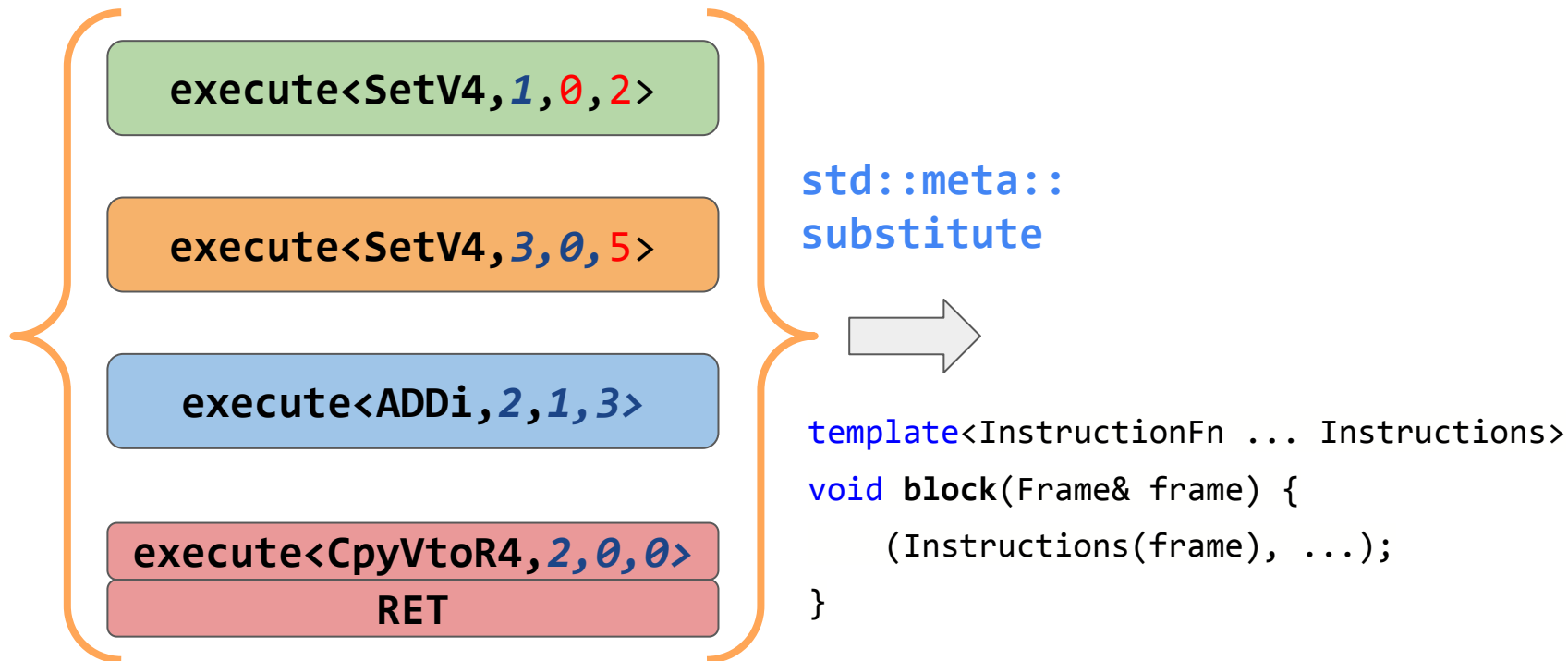
execute<SetV4,3,0,5>

execute<ADDi,2,1,3>

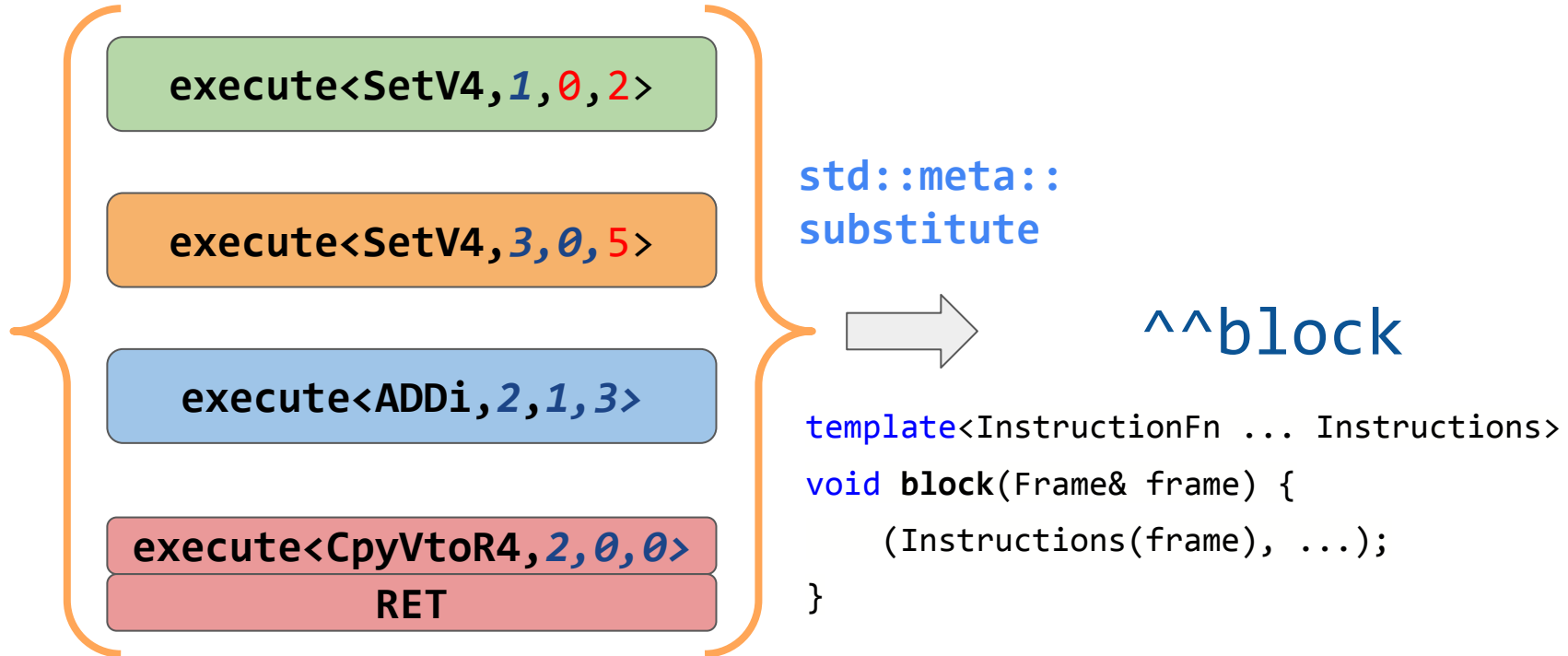
execute<CpyVtoR4,2,0,0>

execute<RET,0,0,0>

Substitute a Bytecode program at compile time



Substitute a Bytecode program at compile time



constexpr

specifies that:

- a **function**
- a **function** template

is

an immediate function

requires that every call to the function is evaluated at compile time.

Consteval function

- A **constexpr** function can use heap allocated memory.
- Must be **transient**
 - cleaned up before the constexpr finishes evaluating.

```
constexpr result = some_consteval_function();
```

- **some_consteval_function** can use heap allocated memory.
- this heap allocated memory must be cleaned up before storing the **result**.

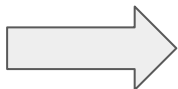
Consteval programming

```
inline constexpr
std::array<dword, 8> sum
{
    0  0x0001004Du,
    1  0x00000002u,
    2  0x0003004Du,
    3  0x00000005u,
    4  0x00020073u,
    5  0x00030001u,
    6  0x00020052u,
    7  0x0000000Au
};
```

Consteval programming

```
inline constexpr
std::array<dword, 8> sum
{
0  0x0001004Du,
1  0x00000002u,
2  0x0003004Du,
3  0x00000005u,
4  0x00020073u,
5  0x00030001u,
6  0x00020052u,
7  0x0000000Au
};
```

Create
offsets



0
2
4
6
7

Consteval programming

```
inline constexpr
std::array<dword, 8> sum
{
0  0x0001004Du,
1  0x00000002u,
2  0x0003004Du,
3  0x00000005u,
4  0x00020073u,
5  0x00030001u,
6  0x00020052u,
7  0x0000000Au
};
```

Create
offsets

0
2
4
6
7



Consteval programming

```
inline constexpr
std::array<dword, 8> sum
{
0  0x0001004Du,
1  0x00000002u,
2  0x0003004Du,
3  0x00000005u,
4  0x00020073u,
5  0x00030001u,
6  0x00020052u,
7  0x0000000Au
};
```

Create
offsets

0
2
4
6
7



```
std::vector<std::meta::info>
instructions;
```

Reflected block specialization

```
template<auto const& ByteCode>
constexpr std::meta::info make_block_info() {
    // Step 1 : create offsets into the Bytecode
```

```
}
```

Reflected block specialization

```
template<auto const& ByteCode>
constexpr std::meta::info make_block_info() {
    // Step 1 : create offsets into the Bytecode

    // Step 2 : use std::meta::substitute to create the function
    // pointers.

}
```

Reflected block specialization

```
template<auto const& ByteCode>
constexpr std::meta::info make_block_info() {
    // Step 1 : create offsets into the Bytecode

    // Step 2 : use std::meta::substitute to create the function
    // pointers.

    // Step 3 : use std::meta::substitute again
    // to create the block function
    return std::meta::substitute(
        ^^block, instructions
    );
}
```

Execute the block

```
inline constexpr std::array<dword, 8> sum
```

```
{
```

```
    0x0001004Du,
```

```
    0x00000002u,
```

```
    0x0003004Du,
```

```
    0x00000005u,
```

```
    0x00020073u,
```

```
    0x00030001u,
```

```
    0x00020052u,
```

```
    0x0000000Au
```

```
};
```

```
int sum(){
```

```
    Frame frame;
```

```
    [: make_block_info<program>() :](frame);
```

```
    return frame.resultValue;
```

```
}
```

Execute the block

```
inline constexpr std::array<dword, 8> sum
```

```
{
```

```
    0x0001004Du,
```

```
    0x00000002u,
```

```
    0x0003004Du,
```

```
    0x00000005u,
```

```
    0x00020073u,
```

```
    0x00030001u,
```

```
    0x00020052u,
```

```
    0x0000000Au
```

```
};
```

```
int sum(){
```

```
    Frame frame;
```

```
    [: make_block_info<program>() :](frame);
```

```
    return frame.resultValue;
```

```
}
```

```
1  main:
```

```
2      mov     eax, 7
```

```
3      ret
```

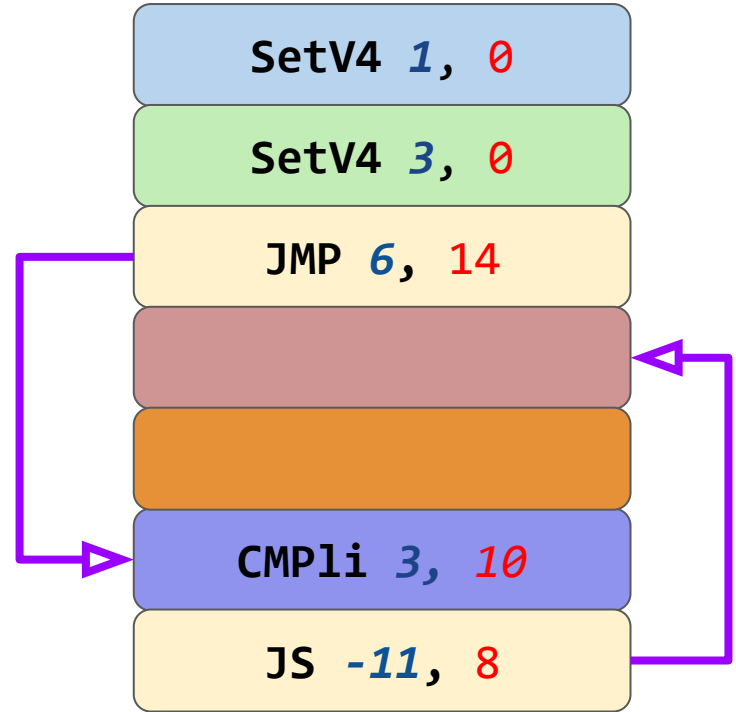
Part 6 - What about jumps and loops?

```
int loop(){  
    int sum = 0;  
    for( int i = 0 ; i < 10 ; ++i )  
    {  
        sum += i;  
    }  
    return sum;  
}
```

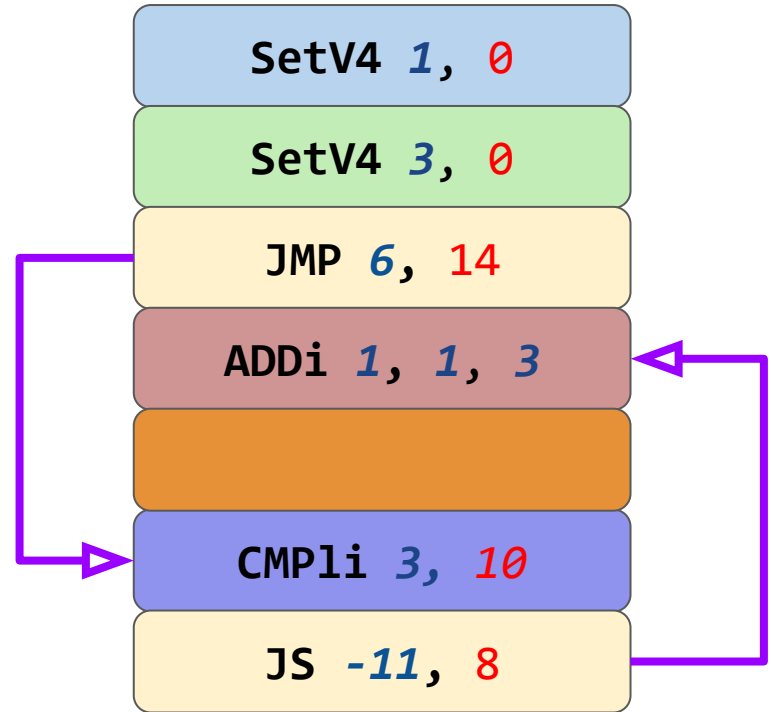
SetV4 1, 0

SetV4 3, 0

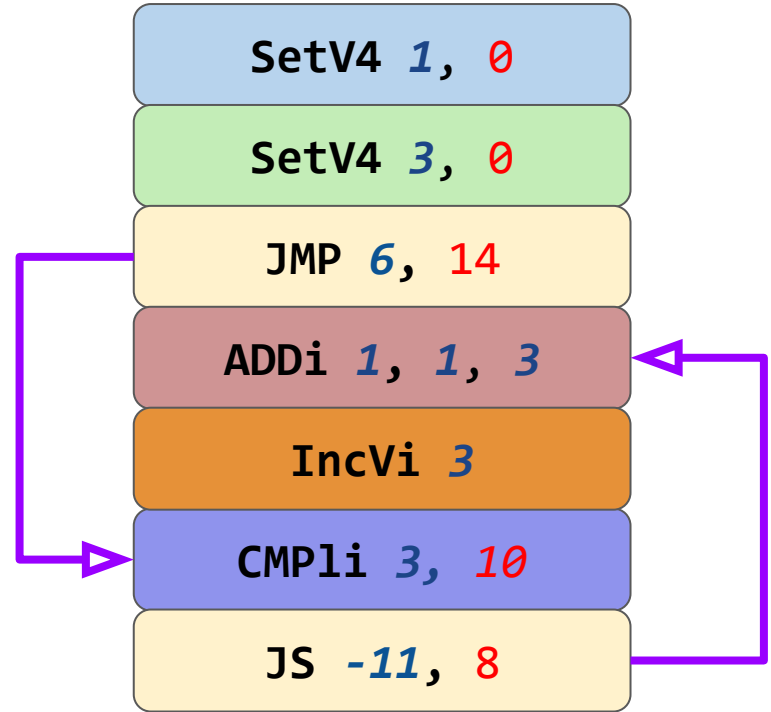
```
int loop(){  
    int sum = 0;  
    for( int i = 0 ; i < 10 ; ++i )  
    {  
        sum += i;  
    }  
    return sum;  
}
```



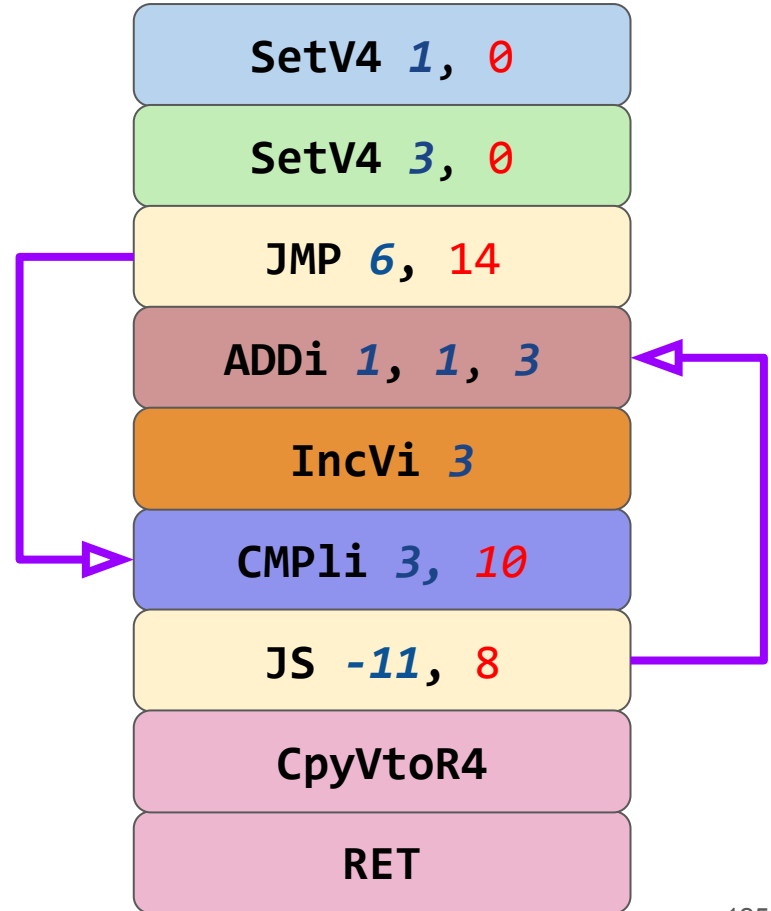
```
int loop(){  
    int sum = 0;  
    for( int i = 0 ; i < 10 ; ++i )  
    {  
        sum += i;  
    }  
    return sum;  
}
```

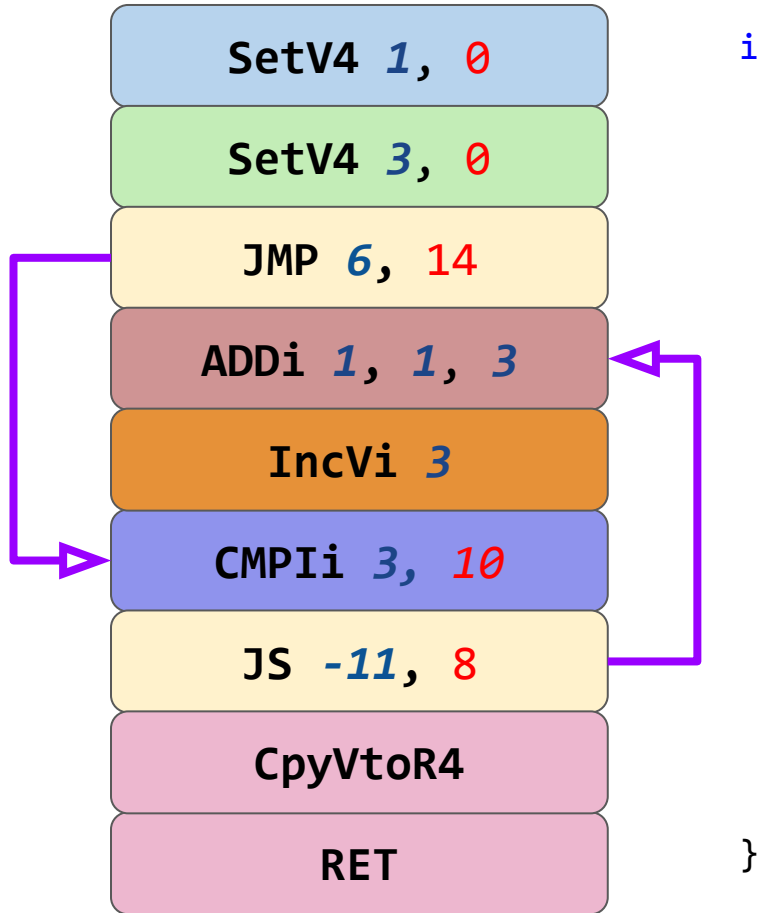


```
int loop(){  
    int sum = 0;  
    for( int i = 0 ; i < 10 ; ++i )  
    {  
        sum += i;  
    }  
    return sum;  
}
```

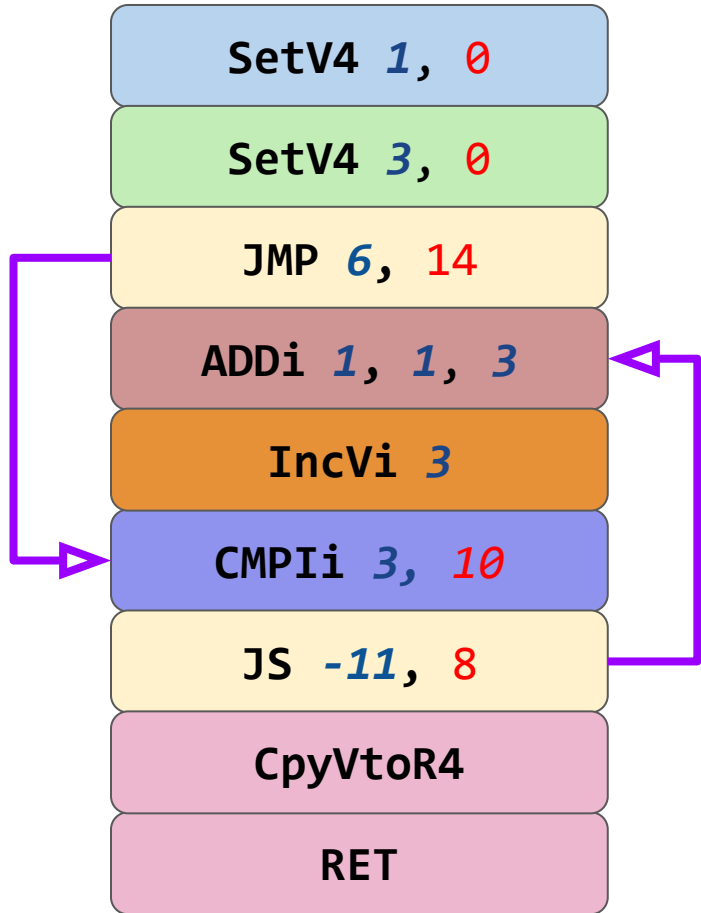


```
int loop(){  
    int sum = 0;  
    for( int i = 0 ; i < 10 ; ++i )  
    {  
        sum += i;  
    }  
    return sum;  
}
```

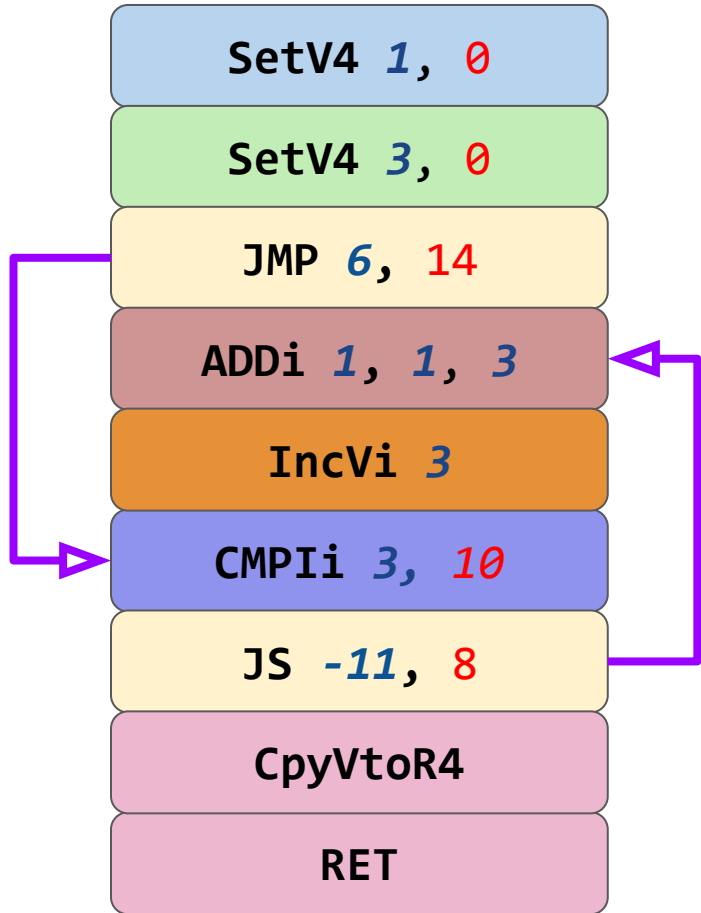




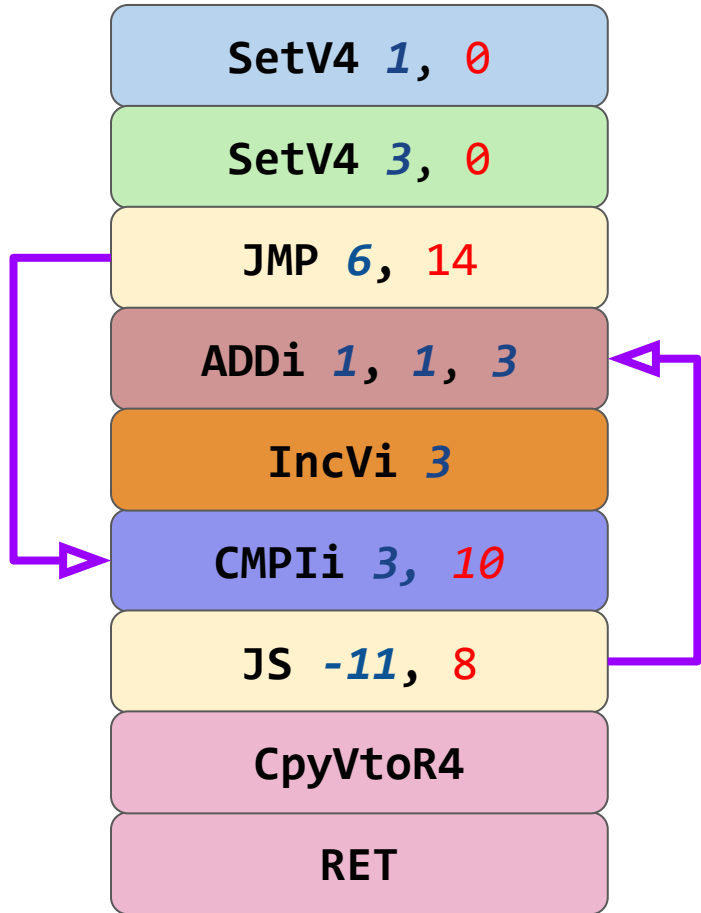
```
int loop() {  
    Frame frame;  
    block <  
        execute<SetV4,1,0,0>  
        execute<SetV4,3,0,0>  
    >(frame);  
}
```



```
int loop() {  
    Frame frame;  
    block <  
        execute<SetV4,1,0,0>  
        execute<SetV4,3,0,0>  
    >(frame);  
    loop <  
  
    >(frame);  
  
}
```



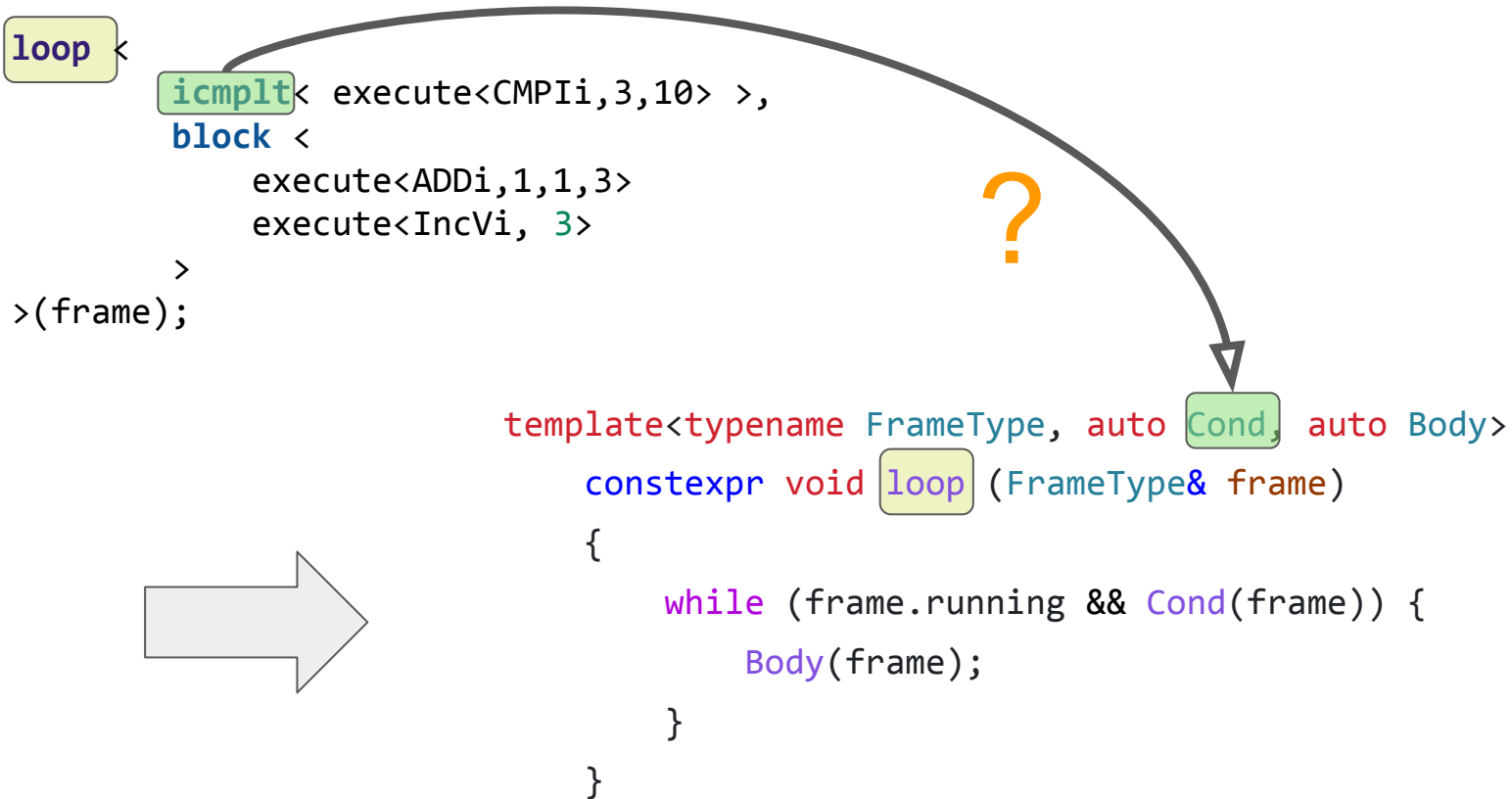
```
int loop() {  
    Frame frame;  
    block <  
        execute<SetV4,1,0,0>  
        execute<SetV4,3,0,0>  
    >(frame);  
    loop <  
        icmplt< execute<CMPIi,3,10> >,  
  
    >(frame);  
  
}
```

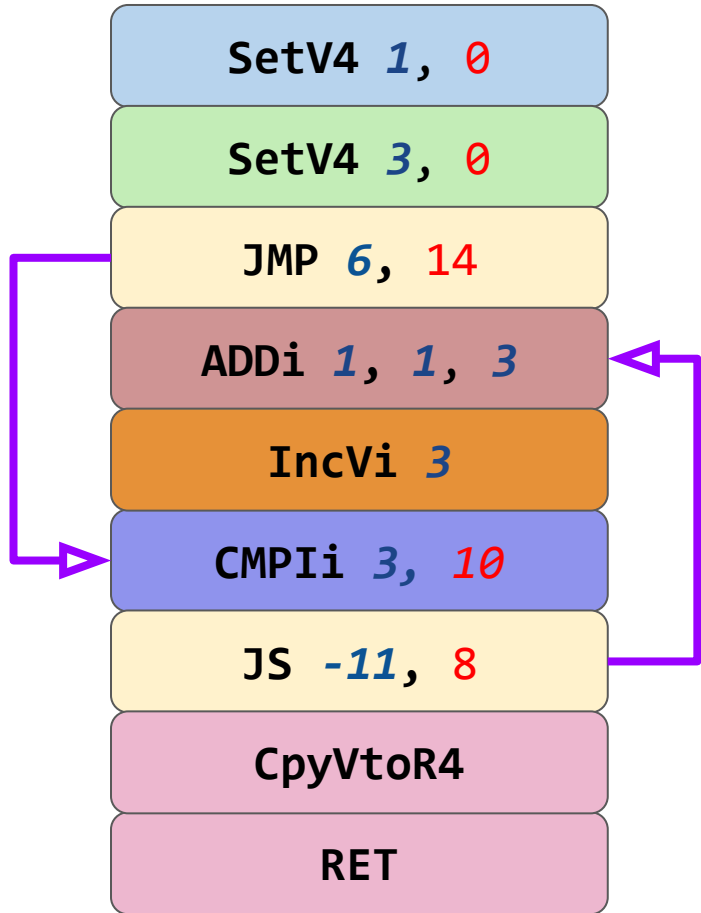


```
int loop() {  
    Frame frame;  
    block <  
        execute<SetV4,1,0,0>  
        execute<SetV4,3,0,0>  
    >(frame);  
    loop <  
        icmplt< execute<CMPIi,3,10> >,  
        block <  
            execute<ADDi,1,1,3>  
            execute<IncVi, 3>  
        >  
    >(frame);  
}
```

```
loop <  
  icmplt< execute<CMPIi,3,10> >,  
  block <  
    execute<ADDi,1,1,3>  
    execute<IncVi, 3>  
  >  
>(frame);
```







```
int loop() {  
    Frame frame;  
    block <  
        execute<SetV4,1,0,0>  
        execute<SetV4,3,0,0>  
    >(frame);  
    loop <  
        icmplt< execute<CMPIi,3,10> >,  
        block <  
            execute<ADDi,1,1,3>  
            execute<IncVi, 3>  
        >  
    >(frame);  
    block <  
        execute<CpyVtoR4,0,0,0>  
        execute<RET,0,0,0>  
    >(frame);  
    return frame.result;  
}
```

```

int loop() {
    Frame frame;
    block <
        execute<SetV4,1,0,0>
        execute<SetV4,3,0,0>
    >(frame);
    loop <
        icmplt< execute<CMPiI,3,10> >,
        block <
            execute<ADDi,1,1,3>
            execute<IncVi, 3>
        >
    >(frame);
    block <
        execute<CpyVtoR4,0,0,0>
        execute<RET,0,0,0>
    >(frame);
    return frame.result;
}

```

```

int loop() {
    Frame frame;
    block <
        execute<SetV4,1,0,0>
        execute<SetV4,3,0,0>
    >(frame);
    loop <
        icmplt< execute<CMPIi,3,10> >,
        block <
            execute<ADDi,1,1,3>
            execute<IncVi, 3>
        >
    >(frame);
    block <
        execute<CpyVtoR4,0,0,0>
        execute<RET,0,0,0>
    >(frame);
    return frame.result;
}

```

1	main:
2	mov eax, 45
3	ret

Part 7 - Function calling

template

<

typename Env

Env: Information about the environment, external function registry, and access to used functions.

template

<

typename Env, auto const& Bytecode,

ByteCode:

an NTTP that contains
the bytecode program

template

<

typename Env, auto const& Bytecode,

ByteCode:

an NTTP that contains
the bytecode program

```
inline constexpr std::array<dword, 8> sum
{
    0x0001004Du,
    0x00000002u,

    0x0003004Du,
    0x00000005u,

    0x00020073u,
    0x00030001u,

    0x00020052u,
    0x0000000Au
};
```

template

<

```
typename Env, auto const& Bytecode,  
typename ReturnType,
```

ReturnType: the actual
ReturnType of the
function (can be void)

template

<

typename Env, auto const& Bytecode,

typename ReturnType, typename... Parameters

>

Parameters: The
Parameters of this
function (can also be
none)

```
template
<
    typename Env, auto const& Bytecode,
    typename ReturnType, typename... Parameters
>
constexpr ReturnType invoke(Parameters... parameters)
{

}
}
```

```
template
```

```
<
```

```
    typename Env, auto const& Bytecode,
```

```
    typename ReturnType, typename... Parameters
```

```
>
```

```
constexpr ReturnType invoke(Parameters... parameters)
```

```
{
```

A. Create a Frame based on the parameters

```
}
```

```
template
```

```
<
```

```
    typename Env, auto const& Bytecode,
```

```
    typename ReturnType, typename... Parameters
```

```
>
```

```
constexpr ReturnType invoke(Parameters... parameters)
```

```
{
```

A. Create a Frame based on the parameters

B. Create the program from constexpr data.

```
}
```

```
template
```

```
<
```

```
    typename Env, auto const& Bytecode,
```

```
    typename ReturnType, typename... Parameters
```

```
>
```

```
constexpr ReturnType invoke(Parameters... parameters)
```

```
{
```

A. Create a Frame based on the parameters

B. Create the program from constexpr data.

C. Execute the bytecode

```
}
```

```
template
```

```
<
```

```
    typename Env, auto const& Bytecode,
```

```
    typename ReturnType, typename... Parameters
```

```
>
```

```
constexpr ReturnType invoke(Parameters... parameters)
```

```
{
```

A. Create a Frame based on the parameters

B. Create the program from constexpr data.

C. Execute the bytecode

D. Return the result if the ReturnType is not a void type.

```
}
```

```
inline constexpr unsigned char demo_asbc[] = { #embed <demo.asbc> };
inline constexpr auto demo_module =
    asbc::format::readSimpleModule<demo_asbc>();

using Environment = asbc::execution_environment<demo_module, HostApi>;

int main()
{
    float result2 = asbc::invoke<
        Environment,
        asbc::format::decodedFunctionByteCode<demo_asbc, 2>,
        float,
        float, float
    >(3.2f, 4.1f);
}
```

Will it fold?

```
int loopTest()
{
    int sum = 0;
    for (int i = 0; i < 10; ++i)
    {
        sum += i;
    }
    return sum;
}
```

Will it fold?

```
constexpr std::int32_t loopTest()  
{  
    return asbc::invoke<asbc::standalone_environment,  
        asbc::format::decodedFunctionByteCode<demo_asbc, 5>, std::int32_t>();  
}
```

```
int loopTest()  
{  
    int sum = 0;  
    for (int i = 0; i < 10; ++i)  
    {  
        sum += i;  
    }  
    return sum;  
}
```

Will it fold?

```
constexpr std::int32_t loopTest()  
{  
    return asbc::invoke<asbc::standalone_environment,  
        asbc::format::decodedFunctionByteCode<demo_asbc, 5>, std::int32_t>();  
}
```

```
int loopTest()  
{  
    int sum = 0;  
    for (int i = 0; i < 10; ++i)  
    {  
        sum += i;  
    }  
    return sum;  
}
```

```
00000000000001040 <main>:  
1040:    f3 0f 1e fa        endbr64  
1044:    b8 2d 00 00 00      mov     eax,0x2d  
1049:    c3                  ret  
104a:    66 0f 1f 44 00 00    nop     WORD PTR  
[rax+rax*1+0x0]
```

Part 8 - Summary

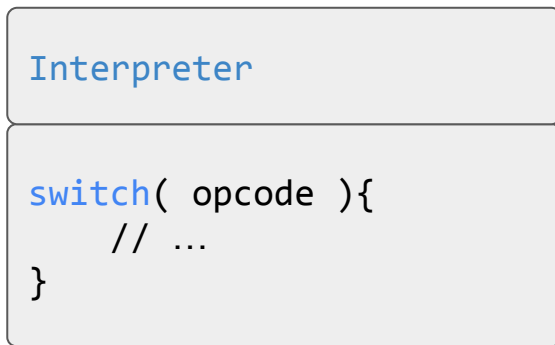
Interpreter mode

```
inline constexpr std::array<dword, 8> sum
{
    0x0001004Du,
    0x00000002u,

    0x0003004Du,
    0x00000005u,

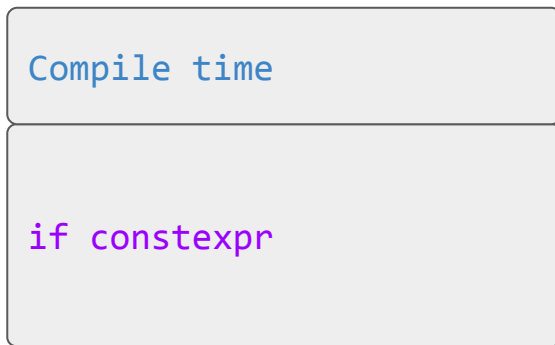
    0x00020073u,
    0x00030001u,

    0x00020052u,
    0x0000000Au
};
```



Handcrafted compile time mode

```
int main(){  
    InstructionFn program = block<  
        execute<SetV4      ,1,0,2>,  
        execute<SetV4      ,3,0,5>,  
        execute<ADDi       ,2,1,3>,  
        execute<CpyVtoR4   ,2,0,0>,  
        execute<RET        ,0,0,0>  
    >;  
  
    Frame frame;  
    program(frame);  
    return frame.R4;  
}
```



Reflected bytecode

```
inline constexpr std::array<dword, 8> sum  
{  
    0x0001004Du,  
    0x00000002u,  
  
    0x0003004Du,  
    0x00000005u,  
  
    0x00020073u,  
    0x00030001u,  
  
    0x00020052u,  
    0x0000000Au  
};
```

std::meta::
substitute



Compile time

if constexpr

- **The approach works**
 - Transform AngelScript bytecode into executable code during C++ compilation
- **The generated code can be optimised**
 - Let the C++ compiler inline instruction calls and eliminate VM overhead
- **The techniques are reusable**
 - Generate template specializations with `std::meta::substitute`
 - Compose instruction calls with fold expressions

- **The approach works**

- Transform AngelScript bytecode into executable code during C++ compilation

- **The generated code can be optimised**

- Let the C++ compiler inline instruction calls and eliminate VM overhead

- **The techniques are reusable**

- Generate template specializations with `std::meta::substitute`
- Compose instruction calls with fold expressions

→ **C++ can serve as a compilation host for other languages**